

Deep Learning Algorithms 2: Interview-Focused Guide

This guide covers modern object detection, vision-language and generative models, graph neural networks, and deep reinforcement learning. Each chapter moves from intuition to equations, implementation, system trade-offs, and placement-style questions.

Contents

1. [YOLO](#)
2. [Faster R-CNN](#)
3. [Mask R-CNN](#)
4. [Vision Transformer](#)
5. [CLIP](#)
6. [Diffusion Models](#)
7. [Stable Diffusion](#)
8. [Graph Neural Network](#)
9. [GCN](#)
10. [GraphSAGE](#)
11. [GAT](#)
12. [DQN](#)
13. [PPO](#)
14. [Actor-Critic](#)

Graph Neural Network

1. Overview

A **Graph Neural Network (GNN)** learns from graph-structured data: nodes, edges, optional attributes, and sometimes a whole-graph label. Most GNNs repeatedly pass messages along edges so each node combines its own state with neighborhood information. They support node classification, edge/link prediction, graph classification/regression, recommendation, fraud detection, molecules, traffic, knowledge graphs, and relational reasoning.

2. Intuition

To judge whether an account is fraudulent, inspect its own features and ask connected accounts for summaries. After one round, it knows immediate neighbors; after two, it indirectly knows neighbors-of-neighbors. The same local rule is reused everywhere, so it works on graphs with different sizes and node orderings.

3. Prerequisites

Graph terminology, adjacency matrices/lists, degree, linear algebra, neural networks, permutation invariance/equivariance, sparse tensors, classification/regression losses, and train/validation/test splitting on relational data.

4. Core Concepts

Concept	Meaning / why it matters	Example	Interview angle
Message passing	Neighbors send learned information over edges	Transaction sends amount/type	General MPNN equation
Aggregation	Permutation-invariant combine: sum/mean/max/attention	Mean neighbor vectors	Why order invariance?
Update	Combines old state with aggregate	MLP/GRU update	Residual and self-loop roles
Readout	Converts node states to graph representation	Sum atoms for molecule property	Node vs graph task
Receptive field	L layers incorporate up to L hops	Two-hop social context	Oversmoothing/oversquashing
Homophily/heterophily	Connected nodes may be similar or dissimilar	Friends vs buyer-seller roles	When classic GCN fails
Transductive/inductive	Same graph nodes vs unseen nodes/graphs	Cite graph vs new users	Which models generalize?

5. Algorithm / Working Process

1. Build graph $G = (V, E)$ with node features x_v , optional edge features e_{uv} , and targets.
2. Initialize hidden states $h_v^{(0)} = x_v$ or learned embeddings.
3. For layer l , compute edge messages from sender, receiver, and edge attributes.
4. Aggregate incoming messages with an order-invariant operator.
5. Update each node, optionally with residual, normalization, dropout, and nonlinearity.
6. Apply a node head, score node pairs for edges, or pool nodes for a graph head.
7. Train with task loss; at inference run full-graph sparse operations or sampled mini-batches.

6. Mathematical Foundation

The message-passing neural network template is

$$m_v^{(l)} = \text{AGG}_{u \in \mathcal{N}(v)} M_l(h_v^{(l)}, h_u^{(l)}, e_{uv}),$$

$$h_v^{(l+1)} = U_l(h_v^{(l)}, m_v^{(l)}).$$

For graph prediction, $h_G = R(\{h_v^{(L)}\})$, where R must be permutation invariant (sum/mean/max or learned pooling). Node classification uses

$$\mathcal{L} = - \sum_{v \in V_{\text{train}}} \sum_c y_{vc} \log \hat{y}_{vc}.$$

Link prediction commonly scores $s(u, v) = h_u^T h_v$ and applies BCE against positive and sampled negative edges. Permuting node order by matrix P should permute node outputs in the same way:

$$f(PX, PAP^T) = Pf(X, A).$$

7. Practical Implementation

```
import torch
from torch import nn

class MeanMessageLayer(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.linear = nn.Linear(2 * in_dim, out_dim)

    def forward(self, x, edge_index):
        src, dst = edge_index
        messages = x[src]
        agg = x.new_zeros(x.shape)
        agg.index_add_(0, dst, messages)
        degree = x.new_zeros(x.size(0))
        degree.index_add_(0, dst, torch.ones_like(dst, dtype=x.dtype))
        agg = agg / degree.clamp_min(1).unsqueeze(1)
        return torch.relu(self.linear(torch.cat([x, agg], dim=1)))

x = torch.tensor([[1., 0.], [0., 1.], [1., 1.], [0., 0.]])
# Include both directions when modeling an undirected graph.
edge_index = torch.tensor([[0, 1, 1, 2, 2, 3],
                           [1, 0, 2, 1, 3, 2]])
layer = MeanMessageLayer(2, 4)
assert layer(x, edge_index).shape == (4, 4)
```

8. Code Explanation

`edge_index[0]` contains senders and row 1 receivers. `index_add_` performs a sparse scatter-sum without a dense adjacency matrix. Dividing by receiver degree gives a mean aggregator; isolated nodes use only their own feature because degree is clamped. Concatenating `x` preserves the center node separately.

9. Training / Evaluation

Define the split based on deployment: temporal splits for future links, cold-start node splits for new users, scaffold splits for new molecular structures, and graph-level splits for independent graphs. Remove validation/test edges from the training message graph when their existence leaks the label. Use macro-F1/ROC-AUC/PR-AUC for node imbalance, Hits@K/MRR for ranking links, MAE/RMSE for regression, and task plus latency/memory. Tune layers, width, dropout, normalization, aggregator, fanout, negative sampling, and edge direction.

10. Complexity and Cost

A basic layer is roughly $O(|E|d + |V|d^2)$, depending on projection order, and stores node/edge activations. Full-batch training is efficient for moderate sparse graphs but impossible for billion-edge graphs. Neighbor sampling reduces batch memory yet causes exponential neighborhood expansion and sampling variance. Distributed partitioning adds communication cost.

11. Common Use Cases

Fraud rings, recommendations, social and citation graphs, molecule properties, protein interactions, traffic forecasting, knowledge graph completion, supply chains, program graphs, and 3-D meshes/point relations.

12. Common Mistakes

- Treating an undirected edge list as bidirectional when it contains one direction only
- Leakage through test edges, future transactions, or graph-derived features
- Random splits that ignore time, entity, scaffold, or connected components
- Negative samples containing unknown positives
- Dense adjacency allocation for a sparse large graph
- Ignoring isolated nodes and zero degrees
- Adding many layers and causing oversmoothing
- Reporting random-split performance for a cold-start product requirement

13. Edge Cases / Limitations

Message passing has limited expressivity related to Weisfeiler-Lehman tests. Deep layers oversmooth representations; narrow graph bottlenecks oversquash many distant signals. Heterophily, dynamic graphs, missing features, noisy edges, hubs, and isolated nodes challenge common architectures. Neighborhood inference may be expensive and predictions can change as the graph evolves.

14. Variations

- **GCN**: normalized neighborhood convolution; foundational placements.
- **GraphSAGE**: sampled inductive aggregation; production graphs.
- **GAT**: learned neighbor attention; important interview variant.
- **GIN**: expressive sum aggregation for graph classification; research/projects.
- **R-GCN/HGT**: relation/type-specific processing for heterogeneous graphs.
- **Graph Transformers**: global/sparse attention with structural encodings; modern research.

15. Related Topics

GNN vs CNN: both share local filters, but graphs have irregular neighborhoods and no fixed ordering. **GNN vs Transformer**: sparse edge-local aggregation versus often global token attention. **Node2Vec vs GNN**: unsupervised lookup embeddings versus feature-aware end-to-end message passing. **Knowledge-graph embeddings**: relation scoring can operate without neighborhood aggregation and often complements GNNs.

16. Interview Questions

1. **What makes data a graph?** Entities are nodes and relationships are edges, possibly with features/types/directions.

2. **What is message passing?** Repeatedly compute edge messages, aggregate by receiver, and update node states.
3. **Why must aggregation be permutation invariant?** Neighbor ordering is arbitrary and should not change the graph function.
4. **What does two GNN layers capture?** Information reachable within up to two message-passing hops.
5. **Transductive vs inductive?** Predict on the known graph/nodes versus generalize to unseen nodes or graphs.
6. **Oversmoothing?** Deep propagation makes node representations increasingly indistinguishable.
7. **Oversquashing?** Exponentially many distant signals are compressed into fixed-width vectors through bottlenecks.
8. **How handle edge features?** Include them in message functions, attention, or relation-specific parameters.
9. **How prevent link-prediction leakage?** Construct train message graph/splits by time and withhold target edges appropriately.
10. **Why neighbor sampling?** It bounds computation and memory for mini-batch learning on large graphs.
11. **When may a GNN be unnecessary?** When relationships add no predictive signal or a tabular baseline already meets requirements.

17. Practice Tasks

Implement sum/mean/max aggregators; run node classification on Cora; create a temporal link split; measure oversmoothing by pairwise cosine similarity with depth; debug an edge-direction error.

18. Project Ideas

Project	Function	Stack / dataset	Resume value
Fraud Ring Detector	Scores accounts and suspicious links	PyG, Neo4j; Elliptic	Relational leakage/imbalance
Molecular Predictor	Predicts molecular properties	PyTorch Geometric; MoleculeNet	Graph-level modeling
Job Recommender	Ranks candidate-job edges	GraphSAGE, FAISS; synthetic/public	Industrial retrieval graphs

19. Quick Revision

- **Idea:** shared local message passing over relations.
- **Formula:** aggregate neighbor messages, then update node state.
- **Use:** relational signal matters.
- **Metrics:** task-dependent plus cold-start/temporal performance.
- **Trap:** graph leakage and incorrect edge direction.
- **One-liner:** A GNN creates node representations by repeatedly aggregating permutation-invariant messages from graph neighborhoods.

20. Final Cheat Sheet

Item	Summary
Input / output	Graph features/edges → node, edge, or graph prediction
Steps	Message → aggregate → update → task readout
Hyperparameters	Layers, width, aggregator, fanout, dropout, negatives
Pros / cons	Exploits relations, reusable / scaling, leakage, oversmoothing
Best use	Data where topology adds predictive information

GCN

1. Overview

A **Graph Convolutional Network (GCN)** propagates node features through a symmetrically normalized adjacency matrix. The canonical layer averages transformed self-and-neighbor features with degree-based normalization. GCN is a foundational semi-supervised node-classification model and a common encoder for links and whole graphs.

2. Intuition

Every node updates its opinion by averaging its own and its neighbors' current opinions, then applying a learned transformation. A popular hub should not overwhelm everyone merely because it has many edges, so normalization reduces contributions according to sender and receiver degrees.

3. Prerequisites

Adjacency/degree matrices, sparse matrix multiplication, self-loops, eigenvalues and graph Laplacian intuition, neural layers, ReLU/dropout, and node classification.

4. Core Concepts

Concept	Meaning / importance	Example	Interview angle
Self-loop	Includes the center node in aggregation	$A + I$	Why preserve own features?
Symmetric normalization	Scales an edge by $1/\sqrt{d_u d_v}$	Hub messages are downweighted	Why not raw adjacency?
Shared weight	Same linear transform at all nodes	$H W$	Parameter sharing/equivariance
Spectral origin	First-order approximation of graph spectral filters	Smooth signals over edges	Spectral vs spatial view
Semi-supervised training	Loss only on labeled train nodes; propagation uses graph	Cora paper labels	Is using unlabeled features allowed?

5. Algorithm / Working Process

1. Add self-loops: $\tilde{A} = A + I$.
2. Compute degrees $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$.
3. Normalize $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$.
4. For each layer, multiply features by trainable weights and propagate with $\hat{A}$, then apply activation/dropout.
5. Use final logits for labeled nodes; for graph prediction, pool final node embeddings.
6. Backpropagate loss through all nodes involved in propagation.

6. Mathematical Foundation

The canonical layer is

$$H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)}).$$

For a node,

$$h_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{h_u^{(l)} W^{(l)}}{\sqrt{\tilde{d}_v \tilde{d}_u}} \right).$$

This performs Laplacian smoothing: connected representations become more similar. A two-layer classifier is $Z = \text{softmax}(\hat{A} \text{ReLU}(\hat{A} X W_0) W_1)$, trained by cross-entropy over the train mask.

7. Practical Implementation

```
import torch
from torch import nn

def normalized_adjacency(a):
    a = a + torch.eye(a.size(0), device=a.device)
    degree = a.sum(1)
    inv_sqrt = degree.clamp_min(1).pow(-0.5)
    return inv_sqrt[:, None] * a * inv_sqrt[None, :]

class GCN(nn.Module):
    def __init__(self, in_dim, hidden, classes):
        super().__init__()
        self.w1 = nn.Linear(in_dim, hidden, bias=False)
        self.w2 = nn.Linear(hidden, classes, bias=False)

    def forward(self, x, a_hat):
        h = torch.relu(a_hat @ self.w1(x))
        return a_hat @ self.w2(h)

a = torch.tensor([[0., 1., 0.], [1., 0., 1.], [0., 1., 0.]])
x = torch.eye(3)
model = GCN(3, 4, 2)
```

```
logits = model(x, normalized_adjacency(a))
assert logits.shape == (3, 2)
```

8. Code Explanation

The function adds self-loops and broadcasts inverse square-root degrees across rows/columns. Each layer first transforms features and then propagates them; propagation/transformation order is interchangeable here by associativity. This dense educational code should become sparse COO/CSR operations for real graphs.

9. Training / Evaluation

For transductive citation benchmarks, apply the loss only on `train_mask` but allow graph propagation across unlabeled nodes if the protocol permits. For inductive claims, withhold test nodes/graphs and rebuild training neighborhoods. Use accuracy/macro-F1, calibration, per-degree analysis, and runtime. Tune 2–3 layers, hidden size, dropout, LR, weight decay, normalization, residuals, and edge preprocessing.

10. Complexity and Cost

Sparse propagation costs roughly $O(|E|d)$; linear projection costs $O(|V|d_{in}d_{out})$. Dense adjacency costs $O(|V|^2)$ memory and is unsuitable for sparse graphs. Full-batch GCN retains all node activations; sampling/partition methods are needed at large scale.

11. Common Use Cases

Citation and product-category node classification, graph-based semi-supervised learning, link encoders, molecule baselines, recommender graph embeddings, and spatial networks.

12. Common Mistakes

- Forgetting self-loops or adding them twice
- Using raw `A` and causing degree-dependent scale explosion
- Normalizing directed graphs as if undirected without deciding semantics
- Materializing dense adjacency
- Applying softmax before `CrossEntropyLoss`
- Counting transductive access as inductive generalization
- Using too many layers without residual/normalization analysis

13. Edge Cases / Limitations

GCN assumes neighboring information should be smoothed, so heterophilous graphs may degrade. Repeated averaging oversmooths. It does not explicitly use edge attributes or distinguish neighbors with identical weighted contributions. Full-batch form is not inherently suited to evolving, unseen-node production graphs.

14. Variations

- **ChebNet**: polynomial spectral filters; historical/research.
- **SGC**: removes intermediate nonlinearities and precomputes propagation; simple large-graph baseline.
- **GCNII**: initial residuals/identity mapping enable deeper GCNs; research.

- **APPNP**: personalized PageRank propagation limits oversmoothing; projects.
- **R-GCN**: relation-specific weights; knowledge graphs.

15. Related Topics

GCN vs GraphSAGE: fixed normalized convolution/full graph versus explicit aggregators and sampling for inductive use. **GCN vs GAT**: degree-fixed weights versus learned attention weights. **GCN vs Laplacian smoothing**: a GCN combines smoothing with learned feature transforms/nonlinearities.

16. Interview Questions

1. **Write the GCN update.** $H' = \text{sigma}(D_{\text{tilde}}^{-1/2} A_{\text{tilde}} D_{\text{tilde}}^{-1/2} H W)$.
2. **Why add self-loops?** To retain/update a node's own information along with neighbors.
3. **Why symmetric normalization?** It controls scale and balances influence by both endpoint degrees.
4. **What is oversmoothing?** Repeated propagation makes connected node embeddings too similar.
5. **Is GCN spectral or spatial?** It was derived spectrally but the layer is implemented as spatial neighborhood aggregation.
6. **Can it process edge features directly?** Not in the canonical formula; extend the message function or use another architecture.
7. **Why usually only two layers on Cora?** More hops can oversmooth and incorporate noisy distant nodes.
8. **How does a GCN handle isolated nodes?** Self-loops ensure a valid degree and preserve their own transformed features.
9. **What leaks in node classification?** Labels/future edges/test-derived graph features; allowed unlabeled topology depends on protocol.
10. **How scale it?** Sparse operations, neighbor/cluster sampling, precomputed propagation, or distributed partitioning.
11. **When will heterophily hurt?** When connected nodes commonly require different labels and averaging destroys discriminative signal.

17. Practice Tasks

Derive the three-node normalized matrix; implement a sparse version; train on Cora; compare raw/row/symmetric normalization; plot embedding similarity versus layer depth.

18. Project Ideas

Project	Function	Stack / dataset	Resume value
Citation Classifier	Predicts paper fields	PyG, Cora/Citeseer	Canonical GCN experiment
Supply Risk Graph	Scores supplier risk propagation	PyTorch, NetworkX; synthetic	Domain graph modeling
Semi-supervised Product Tags	Uses catalog links with few labels	DGL/PyG; Amazon data	Business semi-supervision

19. Quick Revision

- **Idea:** degree-normalized smoothing plus learned transforms.
- **Formula:** $D^{-1/2} (A+I) D^{-1/2} H W$.
- **Use:** homophilous moderate graphs.
- **Metrics:** node task metric, per-degree results, cost.
- **Trap:** leakage, dense adjacency, overdepth.
- **One-liner:** GCN averages self-and-neighbor features with symmetric degree normalization before a learned nonlinear transform.

20. Final Cheat Sheet

Item	Summary
Input / output	Node features + adjacency → node embeddings/logits
Steps	Self-loops → normalize → propagate/project → repeat
Hyperparameters	Layers, hidden size, LR, dropout, weight decay
Pros / cons	Simple, strong baseline / homophily bias, full-batch scaling
Best use	Semi-supervised node tasks on homophilous graphs

GraphSAGE

1. Overview

GraphSAGE (SAmple and aggreGatE) is an inductive GNN framework that learns functions for aggregating sampled neighbor features instead of learning a separate embedding per training node. At inference it can compute representations for unseen nodes from their attributes and neighborhoods. It is important for large, evolving recommendation, fraud, and social graphs.

2. Intuition

When estimating a new user's interests, do not consult millions of neighbors recursively. Sample a fixed number of contacts, summarize them, combine the summary with the user's features, and repeat for a small number of hops. The learned summarizer works even though this user was absent during training.

3. Prerequisites

Message passing, mini-batching, random sampling, adjacency lists, inductive learning, negative sampling, embeddings, and GCN basics.

4. Core Concepts

Concept	Meaning / importance	Example	Interview angle
Inductive function	Embedding computed from features/neighborhood, not node ID only	Embed a new customer	Why can it generalize?

Concept	Meaning / importance	Example	Interview angle
Neighbor sampling	Fixed fanout bounds each batch's computation	Sample 15 then 10 neighbors	Sampling variance/bias
Aggregator	Mean, pooling MLP, or LSTM summarizes neighbors	Mean of sampled neighbor states	Which are permutation invariant?
Concatenation	Keeps self and neighborhood signals distinct	[h_v	
Layer-wise expansion	Requested seeds expand backward by fanouts	512 seeds $\times$ 10 $\times$ 5	Neighborhood explosion

5. Algorithm / Working Process

1. Choose seed nodes for a mini-batch.
2. Sample a bounded fanout of neighbors per seed for the outermost layer, recursively for earlier layers.
3. Initialize sampled nodes from features.
4. From the farthest layer inward, aggregate sampled neighbor states.
5. Concatenate each node's current state with its aggregate, apply shared weights/nonlinearity, optionally normalize.
6. Compute seed-node predictions or pair scores; backpropagate only through the sampled computation graph.
7. At inference, sample or use complete neighborhoods to embed unseen nodes.

6. Mathematical Foundation

For sampled neighbors $S_l(v) \subseteq \mathcal{N}(v)$:

$$h_{\mathcal{N}(v)}^{(l)} = \text{AGG}_l(\{h_u^{(l-1)} : u \in S_l(v)\}),$$

$$h_v^{(l)} = \sigma(W_l[h_v^{(l-1)} \| h_{\mathcal{N}(v)}^{(l)}]), \quad z_v = h_v^{(L)} / \|h_v^{(L)}\|_2.$$

Mean is invariant and unbiased for a uniform neighbor mean, though nonlinear downstream computation introduces variance/bias. With fanouts $s_1, \dots, s_L$, worst-case sampled nodes per seed scale near $\prod_l s_l$. Supervised loss is task-specific; unsupervised GraphSAGE originally used random-walk positive pairs and negative-sampling loss.

7. Practical Implementation

```
import torch
from torch import nn

class SAGEConv(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.linear = nn.Linear(2 * in_dim, out_dim)
```

```
def forward(self, x, edge_index):
    src, dst = edge_index
    neighbor_sum = torch.zeros_like(x).index_add_(0, dst, x[src])
    degree = torch.zeros(x.size(0), device=x.device)
    degree.index_add_(0, dst, torch.ones(src.size(0), device=x.device))
    neighbor_mean = neighbor_sum / degree.clamp_min(1).unsqueeze(1)
    return torch.relu(self.linear(torch.cat([x, neighbor_mean], 1)))

x = torch.randn(5, 8)
edges = torch.tensor([[0,1,1,2,3,4], [1,0,2,1,4,3]])
assert SAGEConv(8, 16)(x, edges).shape == (5, 16)
```

Production code normally uses `torch_geometric.nn.SAGEConv` with `NeighborLoader`, which constructs sampled, relabeled bipartite blocks and avoids computing all nodes.

8. Code Explanation

The layer performs full-neighborhood mean aggregation; sampling changes only which edges enter `edge_index`. Self and neighbor features remain distinguishable through concatenation. Zero-degree nodes receive a zero neighbor vector and can still transform their own features.

9. Training / Evaluation

For a real inductive test, hide test nodes and their incident edges/features during training as appropriate. Use temporal or cold-start splits. Measure task metrics plus sampled/full-neighborhood consistency, variance across sampling seeds, throughput, fanout coverage, and memory. Tune fanout per layer, batch size, depth, dimensions, aggregator, negative sampling, feature normalization, and dropout. Importance or degree-aware sampling may improve coverage of influential neighbors.

10. Complexity and Cost

Sampling bounds computation per seed approximately by the fanout product rather than full graph size. Large fanouts/depth still explode. Sampling and remote feature fetch can dominate GPU compute; graph stores, caching, pinned memory, partitions, and batch construction matter. Inference for many nodes may use layer-wise full-graph evaluation to avoid repeated work.

11. Common Use Cases

Cold-start recommendations, new-account fraud scoring, large social graphs, dynamic product/catalog graphs, inductive node classification, and link prediction.

12. Common Mistakes

- Claiming inductive evaluation while test nodes participated in training propagation
- Sampling after accidentally constructing a leaked future graph
- Assuming LSTM aggregation is permutation invariant without randomized ordering
- Setting large depth/fanout and recreating full-batch cost
- Forgetting bidirectional edges when required
- Comparing stochastic runs with one seed

- Using ID embeddings alone and expecting unseen-node generalization

13. Edge Cases / Limitations

Low fanout may miss rare decisive neighbors; high fanout is expensive. High-degree and low-degree nodes experience different sampling behavior. If unseen nodes lack informative features/neighbors, inductive computation cannot invent identity information. Sampling makes output stochastic and distributed neighborhood retrieval can be a serving bottleneck.

14. Variations

- **Mean GraphSAGE:** simplest scalable default; placement/projects.
- **Pooling aggregator:** MLP per neighbor then max pool; more expressive/costly.
- **LSTM aggregator:** sequence model over neighbors; order issue, mainly historical.
- **PinSAGE:** random-walk importance sampling for web-scale recommendation; industry.
- **Temporal GraphSAGE:** time-aware neighbor sampling/features; dynamic graphs.

15. Related Topics

GraphSAGE vs GCN: sampled inductive aggregation with self/neighbor concatenation versus fixed symmetric normalized convolution. **GraphSAGE vs Node2Vec:** feature-based generalizable encoder versus per-node unsupervised embeddings. **GraphSAGE vs GAT:** uniform/statistical aggregators versus learned neighbor weights.

16. Interview Questions

1. **Why is GraphSAGE inductive?** It learns an aggregation function applied to features, not only embeddings tied to known node IDs.
2. **What problem does sampling solve?** It bounds memory and computation for mini-batch training on large graphs.
3. **What is fanout?** Number of neighbors sampled per node at each layer.
4. **Why concatenate self and neighbor mean?** It preserves their distinct roles before learning how to combine them.
5. **Is mean aggregation permutation invariant?** Yes.
6. **What happens as depth/fanout grows?** Sampled neighborhoods expand roughly multiplicatively.
7. **How test inductive ability?** Hold out complete nodes/graphs or future arrivals, not only their labels.
8. **Can it use edge features?** Not in the simplest layer; extend messages/weights to include them.
9. **How serve embeddings for new nodes?** Fetch features and sampled neighborhoods, then run the learned aggregator.
10. **Why may sampled and full inference differ?** The neighborhood estimate is stochastic and incomplete.
11. **When do ID embeddings hurt?** They do not exist for unseen nodes and can let the model memorize training entities.

17. Practice Tasks

Implement uniform neighbor sampling; compare full vs sampled mean; train on Reddit or OGBN-Arxiv; evaluate held-out nodes; profile data loading versus GPU time.

18. Project Ideas

Project	Function	Stack / dataset	Resume value
Cold-start Recommender	Embeds new users/items from attributes	PyG, FAISS; MovieLens + metadata	Scalable inductive retrieval
Streaming Fraud Scorer	Scores new accounts from recent graph	GraphSAGE, Kafka mock; Elliptic	Temporal production design
Paper Discovery	Recommends papers and embeds new uploads	OGBN-Arxiv, FastAPI	Sampling plus serving

19. Quick Revision

- **Idea:** sample neighbors and learn an inductive aggregator.
- **Formula:** $h' = \sigma(W [h_{self} || AGG(h_{neighbors})])$.
- **Use:** large/evolving graphs and unseen nodes.
- **Metrics:** task metric, sampling variance, throughput.
- **Trap:** false inductive split and fanout explosion.
- **One-liner:** GraphSAGE bounds graph computation through neighbor sampling and generalizes by aggregating features rather than memorizing node embeddings.

20. Final Cheat Sheet

Item	Summary
Input / output	Seed nodes + sampled neighborhoods → embeddings/predictions
Steps	Sample recursively → aggregate → concat self → transform
Hyperparameters	Fanouts, layers, aggregator, batch, width, negatives
Pros / cons	Inductive and scalable / sampling variance and retrieval cost
Best use	Dynamic large graphs and cold-start inference

GAT

1. Overview

A **Graph Attention Network (GAT)** learns a data-dependent importance weight for each edge while aggregating neighbors. Unlike a GCN's degree-determined coefficients, GAT can emphasize relevant neighbors and suppress irrelevant ones. Multi-head attention stabilizes learning and offers multiple relation subspaces.

2. Intuition

When asking colleagues for advice, you should not average all answers equally. For a coding question, a software engineer may receive higher weight than an unrelated contact. GAT learns those weights from the

center and neighbor representations, separately at every layer.

3. Prerequisites

GNN message passing, attention/softmax, adjacency masking, LeakyReLU, multi-head learning, sparse operations, self-loops, and node classification.

4. Core Concepts

Concept	Meaning / importance	Example	Interview angle
Edge attention score	Learned compatibility for connected nodes	Center/neighbor pair score	Additive vs dot-product attention
Neighborhood softmax	Normalizes scores only over incoming neighbors	Weights sum to one per receiver	Which dimension is softmax over?
Multi-head attention	Independent projections/weights are concat or averaged	8 heads $\times$ 8 dims	Concat hidden, average output
Masking	Non-edges cannot exchange messages	Sparse edge list	Complexity still edge-local
Dynamic weights	Coefficients depend on features, not just graph degree	Context-sensitive neighbor relevance	GAT vs GCN

5. Algorithm / Working Process

1. Linearly project node features for each attention head.
2. For each directed edge $u \rightarrow v$, compute an unnormalized compatibility from projected sender and receiver.
3. Apply LeakyReLU and softmax across all incoming edges of receiver v .
4. Weighted-sum sender vectors using normalized coefficients.
5. Concatenate heads in hidden layers or average them in the final layer.
6. Apply activation/dropout/residual/normalization and task head.
7. Optimize normal supervised or self-supervised task loss.

6. Mathematical Foundation

For projected nodes $z_i = Wh_i$, classic GAT computes

$$e_{ij} = \text{LeakyReLU}(a^T[z_i \| z_j]), \quad j \in \mathcal{N}(i),$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}, \quad h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} z_j \right).$$

For K heads, hidden outputs are commonly $\|_{k=1}^K h_i^{(k)}$; final logits often average heads. Attention dropout regularizes edges. Because softmax is per receiver neighborhood, coefficients cannot be compared naively across nodes with different neighborhoods.

7. Practical Implementation

```
import torch
from torch import nn

class DenseGATLayer(nn.Module):
    """Educational single-head GAT; use sparse PyG/DGL for real graphs."""
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.w = nn.Linear(in_dim, out_dim, bias=False)
        self.attn_src = nn.Parameter(torch.empty(out_dim))
        self.attn_dst = nn.Parameter(torch.empty(out_dim))
        nn.init.xavier_uniform_(self.w.weight)
        nn.init.normal_(self.attn_src, std=0.1)
        nn.init.normal_(self.attn_dst, std=0.1)

    def forward(self, x, adjacency):
        z = self.w(x)
        # Row i receives from column j.
        scores = (z @ self.attn_dst)[:, None] + (z @ self.attn_src)[None, :]
        scores = torch.nn.functional.leaky_relu(scores, 0.2)
        scores = scores.masked_fill(~adjacency.bool(), float("-inf"))
        alpha = scores.softmax(dim=1)
        return torch.relu(alpha @ z), alpha

a = torch.tensor([[1,1,0], [1,1,1], [0,1,1]], dtype=torch.bool)
out, attention = DenseGATLayer(4, 8)(torch.randn(3, 4), a)
assert out.shape == (3, 8) and torch.allclose(attention.sum(1), torch.ones(3))
```

8. Code Explanation

The concatenated attention vector is algebraically split into source/destination parameters, avoiding explicit pair concatenation. Non-edge logits become negative infinity before row-wise softmax. Self-loops in `a` prevent empty neighborhoods. This dense $N \times N$ score matrix is pedagogical only; sparse implementations compute scores for listed edges and scatter-softmax by receiver.

9. Training / Evaluation

Use graph-appropriate splits and edge masking as for other GNNs. Tune head count, head dimension, attention/feature dropout, LeakyReLU slope, residuals, LR, and weight decay. Compare against GCN/GraphSAGE under equal hidden dimensions/parameter counts. Evaluate task metrics, calibration, time/memory, and stability over seeds. Treat attention inspection as a hypothesis, not a definitive explanation; test by masking or perturbing high-attention edges.

10. Complexity and Cost

Sparse GAT is roughly $O(|V|d_{in}d_{out} + |E|Kd_{head})$ plus neighborhood softmax. Multiple heads and stored edge coefficients increase memory. Dense naive GAT is $O(|V|^2)$. On large graphs, neighbor sampling is still needed; high-degree softmax/scatter operations may be a bottleneck.

11. Common Use Cases

Node classification with noisy neighbors, molecular/biological graphs, recommendation, traffic networks, heterogeneous relations after extension, and graph problems where neighbor relevance varies by context.

12. Common Mistakes

- Applying softmax over all graph edges instead of per receiver neighborhood
- Failing to mask non-edges before softmax
- Omitting self-loops and producing empty-neighborhood NaNs
- Confusing sender/receiver direction
- Concatenating final heads when the classifier expects averaging
- Claiming attention weights are explanations without intervention
- Building a dense attention matrix for a large sparse graph

13. Edge Cases / Limitations

GAT is more expensive than mean aggregation and can overfit small graphs. Attention can become nearly uniform or overly concentrated. Classic static attention has ranking limitations later addressed by GATv2. It does not solve oversmoothing, oversquashing, or heterophily automatically, and attention scores are not inherently causal or calibrated.

14. Variations

- **GATv2:** changes scoring order for more dynamic attention; projects/research.
- **Relational GAT:** relation/type-aware heads; heterogeneous graphs.
- **Graph Transformer:** global/sparse dot-product attention plus structural encodings; modern research.
- **Edge-feature GAT:** includes edge attributes in score/message; practical graphs.
- **SuperGAT:** attention supervision/regularization; research.

15. Related Topics

GAT vs GCN: learned feature-dependent edge weights versus fixed degree normalization. **GAT vs Transformer attention:** neighborhood-masked additive attention versus commonly global dot-product attention. **GAT vs GraphSAGE:** relevance weighting versus sampled fixed aggregation; they can be combined with sampling.

16. Interview Questions

1. **What does GAT learn?** Attention coefficients determining relative neighbor influence.
2. **Where is softmax applied?** Across each receiver node's allowed incoming neighborhood.
3. **Why multi-head attention?** It stabilizes learning and captures different interaction subspaces.
4. **Concat vs average heads?** Concatenate hidden heads for capacity; average final heads to keep output size/stability.
5. **GAT vs GCN coefficients?** GAT uses node-feature compatibility; GCN uses graph degrees.
6. **Why mask before softmax?** Otherwise non-neighbors receive probability and exchange information.
7. **Does GAT support inductive nodes?** Yes if it uses features and the learned local rule; scaling may require sampling.
8. **Why add self-loops?** Preserve own information and guarantee a valid neighborhood.

9. **Are attention weights explanations?** They may aid inspection but do not establish causal feature/edge importance.
10. **What does GATv2 fix?** It makes attention ranking more query-dependent/expressive than classic static scoring.
11. **Main scaling concern?** Per-edge, per-head coefficients and neighborhood softmax memory/compute.

17. Practice Tasks

Implement sparse edge softmax; verify coefficients sum to one by receiver; train GAT on Cora; compare uniform and learned aggregation; perturb top-attention edges and measure prediction change.

18. Project Ideas

Project	Function	Stack / dataset	Resume value
Traffic Sensor GAT	Forecasts nodes using relevant neighboring roads	PyG Temporal; METR-LA	Spatiotemporal attention
Molecule Attention Explorer	Predicts property and audits substructures	PyG; MoleculeNet	Scientific GNN interpretation
Fraud Neighbor Ranker	Scores accounts with attention over transactions	DGL; Elliptic	Edge features and imbalanced graphs

19. Quick Revision

- **Idea:** learn per-edge neighbor importance.
- **Formula:** neighborhood-softmax of additive compatibility, then weighted sum.
- **Use:** neighbors have unequal relevance.
- **Metrics:** task metric, stability, cost; attention only diagnostically.
- **Trap:** wrong softmax group/direction and dense matrices.
- **One-liner:** GAT replaces fixed graph-normalization weights with learned, neighborhood-normalized attention coefficients.

20. Final Cheat Sheet

Item	Summary
Input / output	Node features + edges → attended node embeddings
Steps	Project → edge score → neighborhood softmax → weighted sum
Hyperparameters	Heads, head dim, dropouts, slope, layers, sampling
Pros / cons	Adaptive neighbors / per-edge cost, unstable interpretation
Best use	Graphs with context-dependent neighbor relevance

Diffusion Models

1. Overview

Diffusion models are generative models that learn to reverse a gradual noising process. Training corrupts real data with Gaussian noise at many noise levels; a neural network learns to predict the added noise, the clean sample, or a related velocity target. Generation begins from random noise and repeatedly denoises it. Diffusion models produce high-quality images, audio, video, molecules, and 3-D content and support conditioning, editing, and inverse problems.

2. Intuition

Suppose a photograph is covered with slightly more static at each step until only noise remains. If a model practices answering “which part of this noisy image is static?” at every step, generation can run the lesson backward: start with static, remove predicted noise a little at a time, and reveal a plausible new image.

3. Prerequisites

Probability distributions, Gaussian noise, conditional probability, Markov chains, KL divergence, score functions, CNN/U-Net, attention, timestep embeddings, gradient descent, and basic stochastic differential equation intuition.

4. Core Concepts

Concept	Meaning / why it matters	Example	Interview angle
Forward process	Fixed Markov chain gradually adds noise	Clean image $\rightarrow$ nearly $\mathcal{N}(\mathbf{0}, \mathbf{I})$	Why is no separate encoder needed?
Reverse process	Learned Gaussian transitions remove noise	$\mathbf{x}_t \rightarrow \mathbf{x}_{t-1}$	What does the network predict?
Noise schedule	Controls signal destroyed per timestep	Linear/cosine betas	How does schedule affect SNR?
Score/noise network	Estimates noise or score of noisy density	U-Net conditioned on t	Relation between epsilon and score
Sampler	Numerical path from noise to data	DDPM, DDIM, DPM-Solver	Quality vs number of steps
Conditioning/guidance	Steers generation using labels/text/measurements	“golden retriever” prompt	Classifier-free guidance trade-off

5. Algorithm / Working Process

Training:

1. Draw a clean sample x_0 , timestep t , and noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
2. Directly construct $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$.
3. Feed x_t , timestep embedding, and optional condition into the denoiser.
4. Compare its prediction with the selected target (often ϵ) and update parameters.

Inference:

1. Sample $x_T \sim \mathcal{N}(0, I)$.
2. For timesteps from T to 1, estimate noise/score and apply a scheduler update.
3. Optionally combine conditional and unconditional predictions for guidance.
4. Return x_0 , clamped/decoded to the data domain.

6. Mathematical Foundation

Choose $\beta_t \in (0, 1)$, $\alpha_t = 1 - \beta_t$, and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. The forward transition and its closed form are

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, \beta_t I),$$

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I).$$

The learned reverse transition is $p_\theta(x_{t-1} | x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_t)$. A widely used simplified objective is

$$\mathcal{L}_{simple} = \mathbb{E}_{x_0, t, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2].$$

For classifier-free guidance,

$$\hat{\epsilon} = \epsilon_\theta(x_t, t, \emptyset) + w[\epsilon_{\theta}(x_t, t, c) - \epsilon_\theta(x_t, t, \emptyset)].$$

Higher w improves condition adherence but can reduce diversity and cause oversaturation. The score satisfies approximately $\nabla_{x_t} \log q(x_t) \propto -\epsilon_\theta(x_t, t) / \sqrt{1 - \bar{\alpha}_t}$.

7. Practical Implementation

```
import torch
from torch import nn
import torch.nn.functional as F

class TinyDenoiser(nn.Module):
    """Educational MLP for 2-D data; image diffusion uses a U-Net/Transformer."""
    def __init__(self, steps=100, hidden=128):
        super().__init__()
        self.time = nn.Embedding(steps, hidden)
        self.net = nn.Sequential(
            nn.Linear(2 + hidden, hidden), nn.SiLU(),
            nn.Linear(hidden, hidden), nn.SiLU(), nn.Linear(hidden, 2),
        )

    def forward(self, x, t):
        return self.net(torch.cat([x, self.time(t)], dim=1))

T = 100
beta = torch.linspace(1e-4, 0.02, T)
alpha_bar = torch.cumprod(1 - beta, dim=0)
model = TinyDenoiser(T)

x0 = torch.randn(256, 2) * 0.2 + torch.tensor([2.0, -1.0])
t = torch.randint(T, (x0.size(0),))
```

```
noise = torch.randn_like(x0)
a = alpha_bar[t, None]
xt = a.sqrt() * x0 + (1 - a).sqrt() * noise
loss = F.mse_loss(model(xt, t), noise)
loss.backward()
print(float(loss))
```

8. Code Explanation

`alpha_bar` stores cumulative signal retention. Advanced indexing chooses one noise level per example, so training covers all timesteps without simulating each previous step. The denoiser receives noisy coordinates plus a learned time embedding and predicts the exact sampled noise. Image models replace the MLP with a multi-resolution U-Net or diffusion Transformer.

9. Training / Evaluation

Normalize data consistently (often `[-1,1]`), sample timesteps uniformly or by loss/SNR importance, and maintain an exponential moving average of weights. Large image models need distributed mixed-precision training and careful gradient/NaN monitoring. Evaluate image quality/diversity with FID/KID and precision-recall; conditional alignment with human evaluation or domain-specific scorers; memorization with nearest-neighbor/deduplication audits. FID depends on sample count and feature domain, so report protocol exactly.

10. Complexity and Cost

Training uses one random timestep per sample and costs roughly one denoiser pass, but useful models require huge datasets and many updates. Inference costs `S` denoiser passes for `S` sampling steps, making it slower than one-pass GAN generation. U-Net activation memory grows with resolution; attention layers can be quadratic in spatial tokens. Distillation, latent diffusion, efficient solvers, and quantization reduce cost.

11. Common Use Cases

Text/image-conditioned synthesis, inpainting/outpainting, super-resolution, deblurring, audio generation, molecular design, trajectory generation, anomaly reconstruction, and posterior sampling for inverse problems.

12. Common Mistakes

- Mixing timestep conventions or scheduler coefficients
- Predicting noise but applying an `x0`/velocity update formula
- Failing to scale data to the model's expected range
- Omitting timestep conditioning
- Comparing FID with different sample counts/preprocessing
- Using excessive guidance and blaming the decoder for artifacts
- Evaluating only attractive cherry-picked samples
- Accidentally adding noise to condition channels that should remain clean

13. Edge Cases / Limitations

Sampling is iterative and expensive. Models can memorize, reproduce data bias, generate invalid structure, and struggle with exact text, counting, global geometry, or strict physical constraints. Likelihood and visual

quality need not agree. Different schedulers can expose training/inference mismatch, and stochastic outputs complicate reproducibility.

14. Variations

- **DDPM:** stochastic ancestral baseline; placement-essential.
- **DDIM:** non-Markovian, optionally deterministic, fewer steps; projects.
- **Score-based SDE:** continuous-time formulation; research.
- **Latent diffusion:** diffuses compressed latents; practical large images.
- **DiT:** Transformer denoiser operating on latent patches; modern research.
- **Consistency/rectified-flow models:** faster few-step generation; emerging research.

15. Related Topics

Diffusion vs GAN: stable likelihood-related denoising and diverse samples versus fast one-pass adversarial generation. **Diffusion vs VAE:** iterative stochastic decoder versus explicit latent-variable reconstruction; latent diffusion uses both. **Score matching:** noise prediction is a reparameterized form of learning the noisy-data score. **Autoregressive models:** sequential tokens versus parallel-state iterative refinement.

16. Interview Questions

1. **What is the forward process?** A fixed Gaussian Markov chain that gradually destroys data structure.
2. **Why can x_t be sampled directly?** Products of Gaussian linear transitions yield the closed-form cumulative distribution.
3. **What does a DDPM usually predict?** The noise added to x_0 , though x_0 and velocity parameterizations are alternatives.
4. **Why condition on timestep?** The denoising task changes with signal-to-noise ratio.
5. **What is classifier-free guidance?** Interpolate/extrapolate between unconditional and conditional denoiser outputs without an external classifier.
6. **Why are diffusion models slow at inference?** Generation requires repeated denoiser evaluations.
7. **DDPM vs DDIM?** DDPM is stochastic ancestral sampling; DDIM can follow a deterministic non-Markovian path with fewer steps.
8. **What is a noise schedule?** The sequence controlling how quickly signal is replaced by noise.
9. **What is latent diffusion?** Apply diffusion in a learned lower-dimensional autoencoder latent rather than pixels.
10. **How evaluate generated images?** FID/KID, diversity/precision-recall, condition alignment, human/domain evaluation, and memorization checks.
11. **Why use EMA weights?** Averaged parameters often yield more stable, higher-quality samples.

17. Practice Tasks

Train the 2-D toy model; implement forward noising visualization; derive x_0 from predicted epsilon; compare DDPM/DDIM step counts; intentionally mix scheduler coefficients and diagnose sample failure.

18. Project Ideas

Project	Function	Stack / dataset	Resume value
CIFAR Denoising Diffusion	Generates 32×32 images from scratch	PyTorch, CIFAR-10	Core derivation and training
MRI Reconstruction	Conditions denoising on undersampled measurements	PyTorch, fastMRI subset	Inverse problems/research
Icon Generator	Fine-tunes a compact latent model on a style	diffusers, LoRA; curated icons	GenAI adaptation/deployment

19. Quick Revision

- **Idea:** learn to reverse gradual Gaussian corruption.
- **Formula:** $x_t = \sqrt{\alpha_{\bar{t}}}x_0 + \sqrt{1-\alpha_{\bar{t}}}\epsilon$.
- **Use:** high-quality conditional generation and inverse problems.
- **Metrics:** FID/KID, alignment, diversity, latency.
- **Trap:** mismatching prediction target and scheduler update.
- **One-liner:** Diffusion generation iteratively converts Gaussian noise into data using a timestep-conditioned denoiser.

20. Final Cheat Sheet

Item	Summary
Input / output	Random noise + optional condition → generated sample
Steps	Forward noising for training; reverse denoising for sampling
Hyperparameters	Schedule, timesteps, target, sampler, guidance, resolution
Pros / cons	Stable, diverse, controllable / slow and compute-heavy
Best use	High-quality generation, editing, inverse tasks

Stable Diffusion

1. Overview

Stable Diffusion is a family of **latent diffusion** text-to-image systems. Instead of denoising full-resolution pixels, it uses a variational autoencoder (VAE) to compress images to spatial latents, a text encoder to represent prompts, and a conditional U-Net or Transformer to denoise the latent. The VAE decodes the final latent into an image. This substantially reduces compute and enables text-to-image generation, image-to-image translation, inpainting, ControlNet, and parameter-efficient personalization.

2. Intuition

Pixel diffusion is like repeatedly editing every tile of a huge mosaic. Stable Diffusion first compresses the mosaic into a smaller blueprint that preserves semantic/spatial content. It denoises the blueprint while consulting a text description, then expands the clean blueprint back into pixels.

3. Prerequisites

Diffusion/DDPM, autoencoders/VAEs, U-Net, CNNs, Transformer text embeddings, cross-attention, classifier-free guidance, schedulers, mixed precision, and prompt/tokenization basics.

4. Core Concepts

Concept	Meaning / why it matters	Example	Interview angle
VAE	Encodes image to lower-resolution latent and decodes it	512×512×3 → 64×64×4	Why latent rather than pixel diffusion?
Text encoder	Maps tokenized prompt to contextual embeddings	CLIP text tower in SD 1.x	Frozen vs trainable conditioning
Conditional denoiser	Predicts latent noise/velocity at timestep	U-Net sees z_t , t , text	Where does conditioning enter?
Cross-attention	Image latent queries attend to text keys/values	"red" influences regions related to "car"	Self- vs cross-attention
Scheduler	Converts model outputs into next latent	Euler, DDIM, DPM-Solver	Scheduler is not the trained model
Guidance	Uses conditional/unconditional predictions	CFG scale 7.5	Fidelity-diversity trade-off
Seed	Initializes random latent	Same settings + seed aid reproducibility	Why outputs can still differ

5. Algorithm / Working Process

Training: encode image x to latent $z_0 = sE(x)$; sample timestep/noise; create z_t ; encode caption; sometimes replace caption with empty text; predict noise/velocity with the conditioned denoiser; optimize MSE-like loss.

Text-to-image inference:

1. Tokenize prompt and negative/empty prompt; compute text embeddings.
2. Sample initial latent noise with requested seed and latent shape.
3. At each scheduler timestep, run denoiser for unconditional and conditional contexts (often batched together).
4. Apply classifier-free guidance and scheduler update.
5. Undo latent scaling, decode through VAE, convert to displayable pixels, and run safety/application checks.

Image-to-image: encode an input image, add noise corresponding to **strength**, then run only the remaining denoising path.

6. Mathematical Foundation

Let $z_0 = sE(x)$, where s matches the latent distribution scale expected during diffusion training. Noise it as

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

The conditional objective is

$$\mathcal{L} = \mathbb{E} \|\epsilon - \epsilon_\theta(z_t, t, T(y))\|_2^2,$$

for text encoder T . Cross-attention with latent queries and text keys/values is

$$\text{softmax} \left(\frac{(ZW_Q)(TW_K)^T}{\sqrt{d}} \right) (TW_V).$$

CFG uses $\hat{\epsilon} = \epsilon_u + w(\epsilon_c - \epsilon_u)$. The VAE has reconstruction plus KL regularization, approximately

$$\mathcal{L}_{VAE} = \|x - D(E(x))\| + \beta D_{KL}(q(z | x) \| \mathcal{N}(0, I)),$$

usually trained separately from the diffusion model.

7. Practical Implementation

```
# pip install diffusers transformers accelerate torch
import torch
from diffusers import StableDiffusionPipeline

model_id = "stable-diffusion-v1-5/stable-diffusion-v1-5"
dtype = torch.float16 if torch.cuda.is_available() else torch.float32
pipe = StableDiffusionPipeline.from_pretrained(model_id, torch_dtype=dtype)
pipe = pipe.to("cuda" if torch.cuda.is_available() else "cpu")

generator = torch.Generator(device=pipe.device).manual_seed(42)
image = pipe(
    prompt="a studio photograph of a small robot reading a book",
    negative_prompt="blurry, distorted, low quality",
    num_inference_steps=30,
    guidance_scale=7.0,
    generator=generator,
    height=512,
    width=512,
).images[0]
image.save("robot-reading.png")
```

8. Code Explanation

The pipeline assembles tokenizer, text encoder, denoiser, scheduler, VAE, and post-processing.

`num_inference_steps` trades speed for convergence but more is not always better. `guidance_scale` controls conditional strength. A seeded `torch.Generator` initializes latent noise; reproducibility also depends on device, precision, library versions, and nondeterministic kernels.

9. Training / Evaluation

For domain adaptation, curate captioned images, remove duplicates/unsafe samples, create subject/style holdouts, and prefer LoRA before full U-Net fine-tuning. Track denoising loss but judge samples with prompt

suites, human preference, CLIP-like alignment only as one signal, FID/KID for matched domains, anatomy/text/counting checks, and memorization audits. Validate multiple seeds per prompt. Tune LR, LoRA rank, caption dropout, resolution/aspect buckets, scheduler, steps, CFG, and image-to-image strength.

10. Complexity and Cost

Latents reduce spatial dimensions, commonly by 8 per side, so core denoising operates on about 1/64 as many spatial positions as pixels. Inference still requires multiple denoiser passes, doubled naively under CFG. VRAM depends on model family, resolution, attention, batch, precision, and VAE decoding. Mixed precision, attention slicing, CPU offload, tiled VAE, distilled models, and quantization trade speed/memory/quality.

11. Common Use Cases

Concept art, product mockups, synthetic training data, inpainting/outpainting, style transfer, controlled generation with pose/depth/edges, personalized subject generation, storyboard and asset ideation.

12. Common Mistakes

- Forgetting required latent scaling before VAE decode/after encode
- Treating scheduler and checkpoint parameterization as interchangeable
- Excessive CFG causing burnt colors and reduced diversity
- Judging a model from one seed or cherry-picked prompts
- Fine-tuning all weights for a small style dataset instead of LoRA
- Ignoring licensing, consent, provenance, safety, and memorization
- Expecting negative prompts to provide hard guarantees
- Using unsupported image dimensions or inconsistent preprocessing

13. Edge Cases / Limitations

Models often fail at readable text, exact counting, hands/anatomy, spatial relations, repeated identities, and strict brand/product geometry. The VAE can lose fine texture or color. Prompt meaning is limited by tokenizer/text encoder. Bias and unsafe content reflect data. Generation is nondeterministic and cannot guarantee factual or legally safe imagery.

14. Variations

- **SD 1.x/2.x:** classic latent U-Net families; placements/projects.
- **SDXL:** larger dual-text-encoder system and refiner option; practical projects.
- **LoRA/DreamBooth/Textual Inversion:** lightweight adapters, subject fine-tuning, or learned token embeddings; project-important.
- **ControlNet/T2I-Adapter:** spatial controls such as pose/depth/edges; projects.
- **Inpainting/img2img:** mask or source-latent conditioning; product features.
- **Turbo/Lightning/LCM:** distilled few-step sampling; deployment.

15. Related Topics

Stable Diffusion vs generic diffusion: a particular latent, text-conditioned architecture/product family versus the broad modeling class. **VAE vs denoiser:** compression/reconstruction versus generative latent refinement.

CLIP vs Stable Diffusion: CLIP-like text embeddings condition generation; CLIP itself retrieves/scores rather than generates. **LoRA vs full fine-tuning:** low-rank weight updates reduce training cost and storage.

16. Interview Questions

1. **Why is Stable Diffusion computationally cheaper than pixel diffusion?** It denoises a spatially compressed VAE latent.
2. **Main components?** Tokenizer/text encoder, conditional denoiser, scheduler, VAE, and safety/application pipeline.
3. **How does text affect the image?** Latent features cross-attend to contextual text embeddings.
4. **What is CFG?** Combine conditional and unconditional predictions to strengthen prompt adherence.
5. **What does negative prompt mean?** It supplies the unconditional/negative conditioning branch; it is guidance, not a strict exclusion rule.
6. **What is image-to-image strength?** The starting noise level/portion of denoising, controlling preservation versus change.
7. **Why LoRA?** It trains compact low-rank updates, reducing VRAM, time, and checkpoint size.
8. **What does the scheduler do?** It defines timesteps and numerical updates from model predictions to the next latent.
9. **Why can the same seed differ?** Hardware, precision, scheduler, dimensions, software, prompt processing, or nondeterministic operations changed.
10. **What is ControlNet?** A trainable control branch that injects spatial conditions while preserving a pretrained diffusion backbone.
11. **Major responsible-AI checks?** Data/license provenance, consent, bias, unsafe output, watermark/provenance policy, and memorization.

17. Practice Tasks

Reproduce an image with fixed settings; sweep CFG and steps; implement img2img; train a small LoRA; diagnose a scheduler/prediction-type mismatch; build a prompt/seed evaluation grid.

18. Project Ideas

Project	Function	Stack / dataset	Resume value
Controlled Interior Designer	Generates rooms from edge/depth layout	diffusers, ControlNet, FastAPI	Controllable GenAI product
Brand-Safe Asset Studio	LoRA style adaptation plus prompt/evaluation suite	SDXL, PEFT, MLflow; licensed assets	Fine-tuning and governance
Synthetic Defect Lab	Creates masked defects for detector augmentation	inpainting, OpenCV; MVTec AD	GenAI-to-CV evaluation

19. Quick Revision

- **Idea:** text-conditioned diffusion in VAE latent space.
- **Formula:** latent noise-prediction MSE plus CFG at sampling.
- **Use:** affordable high-resolution generation/editing.
- **Metrics:** human preference, alignment, FID/KID, latency, safety.

- **Trap:** scheduler/latent scaling mismatch and over-guidance.
- **One-liner:** Stable Diffusion denoises compressed image latents under cross-attentive text conditioning, then decodes them with a VAE.

20. Final Cheat Sheet

Item	Summary
Input / output	Prompt + noise/optional image/mask → image
Steps	Text encode → latent denoise → VAE decode
Hyperparameters	Seed, steps, scheduler, CFG, size, strength, LoRA scale
Pros / cons	Flexible, cheaper than pixels / iterative, biased, imperfect control
Best use	Text/image-guided synthesis and editing

YOLO

1. Overview

YOLO (You Only Look Once) is a family of one-stage object detectors. A single network maps an image directly to bounding boxes, objectness scores, and class probabilities. Unlike two-stage detectors, YOLO does not first generate region proposals and then classify them, so it is widely used when latency matters: video analytics, robotics, autonomous driving, retail, and edge devices.

Modern YOLO releases differ substantially in backbone, label assignment, loss, and whether they use anchors, but retain the same engineering idea: dense, end-to-end detection in one forward pass.

2. Intuition

Imagine placing grids at several resolutions over an image. Each grid location asks: "Is an object centered near me? If yes, what class is it and how far are its four boundaries?" Fine feature maps find small objects; coarse maps find large objects. All locations answer simultaneously, making YOLO fast.

3. Prerequisites

- CNNs, feature maps, stride, receptive fields, and transfer learning
- Bounding-box formats (**xyxy**, **xywh**) and Intersection over Union (IoU)
- Binary/multiclass cross-entropy, regression loss, and backpropagation
- Precision, recall, non-maximum suppression (NMS), and mean AP
- PyTorch tensors and image augmentation

4. Core Concepts

Concept	Meaning and importance	Simple example	Interview angle
One-stage detection	Dense classification and box regression without an explicit proposal stage; enables low latency	Predict at every cell of three feature maps	Why is YOLO faster than Faster R-CNN?
Multi-scale features	A feature pyramid predicts at different strides	Stride 8 for a small ball, stride 32 for a bus	How are small objects handled?
Objectness	Probability that a candidate corresponds to an object	A cell on background should have low objectness	Distinguish objectness from class probability
Anchors / anchor-free heads	Anchors regress offsets from preset shapes; anchor-free heads predict centers/distances directly	A tall anchor fits a person	Why do newer detectors avoid anchors?
Label assignment	Chooses which prediction locations are positive for each ground-truth object	Assign nearby high-IoU cells to a box	Static vs dynamic matching
NMS	Removes duplicate boxes with large overlap	Keep score .93, suppress overlapping score .81	When can NMS remove a true positive?

5. Algorithm / Working Process

1. **Input:** resize/letterbox an RGB image, normalize it, and form a tensor $[B, 3, H, W]$.
2. **Backbone:** extract hierarchical visual features.
3. **Neck:** fuse shallow spatial detail with deep semantic features, often using FPN/PAN-style paths.
4. **Detection head:** at multiple scales predict box coordinates, object confidence, and class logits.
5. **Training:** match ground-truth boxes to predictions and minimize box, objectness, and classification losses.
6. **Inference:** decode coordinates, remove low-confidence boxes, apply class-aware or class-agnostic NMS, and map boxes back to the original image.
7. **Output:** $[x1, y1, x2, y2, score, class]$ for each retained detection.

6. Mathematical Foundation

For boxes A and B :

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

A common confidence is $s = p(\text{object})p(c \mid \text{object})$. A simplified training objective is

$$\mathcal{L} = \lambda_{box} \mathcal{L}_{IoU} + \lambda_{obj} \text{BCE}(o, \hat{o}) + \lambda_{cls} \text{BCE}(y, \hat{y}).$$

IoU losses such as GIoU/DIoU/CIoU provide a gradient even when boxes do not overlap. CIoU additionally penalizes center distance and aspect-ratio mismatch. Focal-style classification loss

$$\mathcal{L}_{focal} = -\alpha(1 - p_t)^\gamma \log p_t$$

reduces the influence of easy background examples. During NMS, candidates are sorted by score; boxes whose IoU with a retained box exceeds threshold τ are suppressed.

7. Practical Implementation

```
# pip install ultralytics
from ultralytics import YOLO

# Transfer learning from pretrained weights.
model = YOLO("yolo26n.pt")
model.train(data="data.yaml", epochs=30, imgsz=640, batch=16)

# Inference returns boxes, confidences, and classes after NMS.
result = model("test.jpg", conf=0.25, iou=0.45)[0]
for xyxy, score, cls in zip(
    result.boxes.xyxy.cpu(), result.boxes.conf.cpu(), result.boxes.cls.cpu()
):
    print(xyxy.tolist(), float(score), model.names[int(cls)])
```

`data.yaml` contains dataset paths and class names:

```
path: datasets/traffic
train: images/train
val: images/val
names: [car, bus, pedestrian]
```

8. Code Explanation

`YOLO("yolo11n.pt")` loads a small pretrained checkpoint; transfer learning needs far less labeled data than training from scratch. `train` handles augmentation, matching, loss computation, validation, and checkpointing. At inference, `conf` removes weak candidates and `iou` controls NMS aggressiveness. Coordinates are in the original image space.

9. Training / Evaluation

Split by scene, camera, patient, or video—not random frames—to prevent leakage. Preserve rare classes across splits. Use geometric and photometric augmentation; verify transformed boxes visually. Report AP at IoU .50, stricter `AP@[.50:.95]`, per-class AP, recall, latency, and model size. Tune image size, batch size, learning rate, weight decay, augmentation strength, confidence, and NMS IoU. Diagnose underfitting with low train/validation AP and overfitting with rising train AP but falling validation AP.

10. Complexity and Cost

CNN cost is dominated by convolutions, approximately $O(HWC_{in}C_{out}K^2)$ per layer. Cost grows roughly quadratically with image side length. Small YOLO variants can run in real time on a GPU or optimized edge

accelerator; larger variants improve AP but need more VRAM and latency. Batch inference increases throughput but not per-image latency. Post-processing may become a bottleneck when thousands of candidates survive.

11. Common Use Cases

- Traffic participants and safety-gear detection
- Defect, crop, animal, and medical-region localization
- Sports tracking and retail shelf analytics
- Robot perception and mobile/edge inspection

12. Common Mistakes

- Using wrong normalized/absolute box coordinates or class IDs
- Randomly splitting adjacent video frames, causing leakage
- Treating accuracy as a detection metric instead of AP/recall
- Training at a resolution that erases small targets
- Ignoring class imbalance or empty images
- Tuning confidence/NMS on the test set
- Applying image augmentation without transforming boxes

13. Edge Cases / Limitations

YOLO struggles with tiny, crowded, heavily occluded, unusual-domain, or motion-blurred objects. NMS can suppress adjacent instances of the same class. Predictions are not inherently temporally consistent. Letterboxing and extreme aspect ratios can waste computation. High confidence does not guarantee calibrated probabilities, and open-set objects may be confidently misclassified.

14. Variations

Variation	What changes / when to use	Importance
YOLOv3/v4/v5	Anchor-based historical families; useful for understanding evolution and legacy projects	Placement
YOLOX / newer YOLO heads	Anchor-free, decoupled heads and improved assignment	Projects/placement
YOLO segmentation/pose	Adds mask coefficients or keypoints to the head	Projects
Tiny/nano variants	Reduced depth/width for edge latency	Deployment
NMS-free detectors	One-to-one assignment removes post-processing	Research

15. Related Topics

YOLO vs Faster R-CNN: speed-first dense prediction versus proposal-based, often higher-precision detection.

YOLO vs SSD/RetinaNet: all are one-stage; RetinaNet popularized focal loss. **Detection vs segmentation:**

boxes provide coarse localization; masks identify pixels. **Tracking:** a detector supplies observations to motion/association algorithms such as ByteTrack.

16. Interview Questions

1. **Why is YOLO called one-stage?** It predicts classes and boxes densely in one network pass without a separate proposal classifier.
2. **Objectness vs class probability?** Objectness estimates whether an object exists; class probability identifies its category conditional on existence.
3. **Why multi-scale heads?** Different feature strides offer resolution/semantics suited to different object sizes.
4. **What does NMS do?** It retains high-score boxes and suppresses highly overlapping duplicates.
5. **Why use IoU loss instead of coordinate MSE?** IoU aligns optimization with box overlap and is less sensitive to box scale.
6. **How do you improve small-object recall?** Increase resolution, preserve fine feature maps, tile images, improve labels, and tune assignment/augmentation.
7. **What is anchor-free detection?** It predicts centers, corners, or boundary distances without preset anchor shapes.
8. **What is mAP?** Mean average precision: AP integrated over a precision-recall curve and averaged across classes/IoU thresholds.
9. **Why can lower NMS IoU hurt crowded scenes?** Nearby true instances overlap and one may be suppressed.
10. **How would you deploy YOLO?** Export to ONNX/TensorRT/CoreML, use fixed shapes and appropriate precision, then benchmark end-to-end latency and AP.
11. **Main source of train/production failure?** Domain shift in cameras, weather, object scale, and label definitions.

17. Practice Tasks

- Implement IoU and NMS in NumPy.
- Train a three-class traffic detector and report per-class **AP50** and **AP50-95**.
- Compare 320, 640, and 960-pixel training for accuracy/latency.
- Debug deliberately shifted bounding-box annotations using an overlay viewer.
- Extend detection with ByteTrack and measure track stability.

18. Project Ideas

Project	What it does	Stack / dataset	Resume value
PPE Guardian	Detects helmets/vests and raises zone alerts	PyTorch, YOLO, FastAPI; Roboflow PPE	End-to-end CV deployment
Traffic Edge Monitor	Counts vehicles on an edge device	YOLO, OpenCV, ONNX/TensorRT; UA-DETRAC	Optimization and tracking
Defect Inspector	Finds manufacturing defects with human review	YOLO, MLflow; NEU-DET/custom data	Imbalance, MLOps, explainable errors

19. Quick Revision

- **Key idea:** dense, one-pass, multi-scale object detection.
- **Main formula:** `box loss + objectness loss + class loss`; evaluate with IoU/AP.
- **When to use:** low-latency closed-set detection.
- **Metrics:** `AP50-95`, recall, per-class AP, latency, FPS.
- **Trap:** leakage from adjacent frames and incorrect boxes.
- **One-liner:** YOLO trades a proposal stage for dense end-to-end predictions, making detection fast.

20. Final Cheat Sheet

Item	Summary
Definition	One-stage dense object detector
Input / output	Image → boxes, scores, classes
Steps	Backbone → neck → heads → decode → NMS
Hyperparameters	Image size, LR, loss weights, confidence/NMS thresholds
Metrics	mAP, recall, latency, model size
Pros / cons	Fast and deployable / weaker on tiny, crowded, shifted data
Best use	Real-time detection on server or edge

Faster R-CNN

1. Overview

Faster R-CNN is a two-stage detector: a **Region Proposal Network (RPN)** proposes likely object regions, then an ROI head classifies and refines them. It introduced shared convolutional features for proposal generation and detection, replacing slow external selective search. It is a strong accuracy-oriented baseline for autonomous perception, document analysis, medical imaging, and research.

2. Intuition

Instead of asking every image position to make a final decision immediately, Faster R-CNN first asks a scout to shortlist promising rectangles. A specialist then examines each shortlist item carefully and assigns a class plus a more accurate box.

3. Prerequisites

CNN backbones, anchors, IoU, classification/regression loss, NMS, feature pyramids, bilinear sampling, transfer learning, and detection metrics.

4. Core Concepts

Concept	Meaning / why it matters	Example	Interview angle
RPN	Sliding head predicts anchor objectness and offsets	Propose 1,000 candidate regions	How is it trained jointly?
Anchor	Reference box at a feature location	3 scales $\times$ 3 aspect ratios	Anchor coverage and imbalance
ROI Align	Samples a fixed feature grid without harsh coordinate rounding	Convert variable ROI to $7 \times 7 \times C$	ROI Pool vs ROI Align
ROI head	Classifies proposals and performs class-specific/agnostic regression	"dog" + refined boundaries	Why a second regression?
FPN	Combines feature levels for scale robustness	Small object uses P2/P3	How is an ROI assigned to a level?

5. Algorithm / Working Process

1. Input image is normalized and passed through a backbone/FPN.
2. At every pyramid location, the RPN scores anchors and predicts four offsets.
3. Decode anchors, clip boxes, remove tiny boxes, apply NMS, and retain top proposals.
4. Match proposals to ground truth; ROI Align extracts a fixed-size tensor per proposal.
5. The box head outputs $K+1$ class logits and refined box offsets.
6. During training, optimize RPN objectness/box losses and ROI classification/box losses jointly.
7. During inference, remove background/low scores and apply per-class NMS.

6. Mathematical Foundation

Given anchor (x_a, y_a, w_a, h_a) and target box (x, y, w, h) , regression targets are

$$t_x = (x - x_a)/w_a, \quad t_y = (y - y_a)/h_a, \quad t_w = \log(w/w_a), \quad t_h = \log(h/h_a).$$

The multitask loss is

$$\mathcal{L} = \mathcal{L}_{rpn_cls} + \lambda_1 \mathcal{L}_{rpn_box} + \mathcal{L}_{roi_cls} + \lambda_2 \mathcal{L}_{roi_box},$$

where classification uses cross-entropy and positive anchors/proposals use Smooth L1:

$$\text{smooth}_{L1}(d) = \begin{cases} d^2/(2\beta), & |d| < \beta \\ |d| - \beta/2, & \text{otherwise.} \end{cases}$$

Anchors are labeled positive/negative using IoU thresholds; unmatched anchors are ignored. Mini-batch sampling controls the large foreground/background imbalance.

7. Practical Implementation

```
import torch
from torchvision.models.detection import (
    FasterRCNN_ResNet50_FPN_Weights,
    fasterrcnn_resnet50_fpn,
```

```

)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = fasterrcnn_resnet50_fpn(
    weights=FasterRCNN_ResNet50_FPN_Weights.DEFAULT
).to(device)
model.eval()

# torchvision detection models receive a list of [C,H,W] float images.
image = torch.rand(3, 640, 640, device=device)
with torch.no_grad():
    pred = model([image])[0]
    keep = pred["scores"] > 0.7
    print(pred["boxes"][keep], pred["labels"][keep], pred["scores"][keep])

# Training target for one image:
target = {
    "boxes": torch.tensor([[50., 70., 280., 400.]], device=device),
    "labels": torch.tensor([1], dtype=torch.int64, device=device),
}
model.train()
losses = model([image], [target])
loss = sum(losses.values())
loss.backward()

```

8. Code Explanation

The pretrained model includes a ResNet-50 FPN, RPN, ROI Align, and box head. Evaluation returns decoded post-NMS detections. Training mode with targets returns a dictionary of differentiable RPN and ROI losses. For a custom K-class dataset, replace the final predictor with `FastRCNNPredictor(in_features, K + 1)` because index 0 is background.

9. Training / Evaluation

Validate annotation order `x1<x2, y1<y2`; split by independent source; use detection-safe augmentation. Start from COCO weights and lower the learning rate for the backbone. Monitor each loss independently: falling classification loss with stagnant RPN recall suggests proposal/anchor problems. Report COCO mAP, AP by object size, proposal recall, latency, and peak VRAM. Tune anchor sizes, aspect ratios, proposals per image, ROI batch size, positive fraction, and NMS thresholds.

10. Complexity and Cost

Backbone/FPN cost is image-wide; ROI-head cost grows with the number of proposals. Training stores multi-level features and sampled ROIs, so memory is higher than many one-stage models. Inference is commonly tens rather than hundreds of FPS. RPN NMS and per-class NMS add variable cost. Mixed precision reduces memory, but box/NMS operations should be numerically checked.

11. Common Use Cases

High-accuracy object detection, sparse-object medical images, document/layout analysis, wildlife monitoring, and research baselines where latency is secondary.

12. Common Mistakes

- Forgetting the background class when replacing the predictor
- Supplying integer or wrongly scaled images/boxes
- Including degenerate boxes or wrong label dtypes
- Using default anchors for radically different object scales
- Assuming RPN proposals are class-specific
- Evaluating only **AP50** or only loss
- Randomly splitting related images

13. Edge Cases / Limitations

It is slower and more memory-intensive than one-stage detectors. Poor anchor coverage harms elongated/tiny objects. RPN and ROI sampling can miss extreme class imbalance. NMS struggles with crowded instances. Fixed-resolution ROI features lose detail, and performance drops under domain shift.

14. Variations

- **Fast R-CNN**: uses external proposals; historically important.
- **Cascade R-CNN**: sequential heads use increasing IoU thresholds for high-quality localization; projects/research.
- **DetectoRS / deformable backbones**: improve features at greater cost; research.
- **Class-agnostic regression**: one box regressor for all classes; useful with limited data.
- **Faster R-CNN with alternative backbones**: MobileNet for cost, ResNeXt/Swin for accuracy.

15. Related Topics

Faster R-CNN vs YOLO: proposal-based accuracy and flexibility versus throughput. **RPN vs selective search**: learned, shared, GPU-friendly proposals versus hand-designed CPU proposals. **ROI Align vs ROI Pool**: continuous bilinear sampling versus quantized bins. **Mask R-CNN**: adds a parallel instance-mask head.

16. Interview Questions

1. **What made Faster R-CNN “faster”?** It replaced external proposal generation with a learned RPN sharing backbone features.
2. **Are RPN proposals class-specific?** No; they estimate generic objectness and box offsets.
3. **Why anchors?** They provide scale/aspect-ratio reference boxes for local regression.
4. **Why two regressions?** RPN refines anchors into proposals; ROI head refines proposals for final class-aware localization.
5. **What is ROI Align?** Bilinear sampling of continuous ROI coordinates into a fixed grid.
6. **How are anchors labeled?** By IoU thresholds and usually the best anchor for every ground-truth box.
7. **Why sample negatives?** Background anchors vastly outnumber positives and would dominate the loss.
8. **Role of FPN?** It gives semantically strong features at multiple resolutions.
9. **How do you debug low RPN recall?** Inspect anchor coverage, label matching, proposal counts, NMS, and ground-truth scale.

10. **Why is inference slower than YOLO?** Per-proposal ROI extraction/head processing and multiple NMS stages add work.
11. **Can it train end-to-end?** Yes, shared features and all differentiable components except discrete selection are optimized from the combined losses.

17. Practice Tasks

Implement anchor encoding/decoding; visualize RPN proposals; fine-tune on Penn-Fudan pedestrians; compare anchor settings; debug a dataset containing zero-area boxes.

18. Project Ideas

Project	Function	Stack / dataset	Resume value
Document Element Detector	Finds tables, figures, titles	torchvision, PubLayNet	Layout/CV pipeline
Wildlife Counter	Detects rare animals in camera traps	PyTorch, W&B; Snapshot Serengeti	Long-tail evaluation
Lesion Localizer	Localizes suspicious regions	MONAI/torchvision; public medical data	High-recall, domain-specific CV

19. Quick Revision

- **Idea:** RPN proposals followed by ROI classification/refinement.
- **Formula:** four-term RPN + ROI multitask loss.
- **Use:** accuracy-oriented detection.
- **Metrics:** mAP, proposal recall, latency.
- **Trap:** wrong boxes/labels and unsuitable anchors.
- **One-liner:** Faster R-CNN shares a backbone between learned proposals and a precise second-stage detector.

20. Final Cheat Sheet

Item	Summary
Input / output	Image → scored class boxes
Main steps	Backbone/FPN → RPN → NMS → ROI Align → box head → NMS
Hyperparameters	Anchors, IoU thresholds, proposal count, ROI samples, LR
Pros / cons	Accurate, flexible / slower, more memory
Best use	Server-side, accuracy-first detection

Mask R-CNN

1. Overview

Mask R-CNN extends Faster R-CNN with a parallel head that predicts a binary mask for every detected instance. It performs **instance segmentation**: it distinguishes separate objects and assigns each object its own pixel mask. Applications include medical boundaries, autonomous perception, robotics grasping, satellite imagery, and image editing.

2. Intuition

Faster R-CNN draws a rectangle around each person. Mask R-CNN additionally colors the exact pixels belonging to each person—even when two people overlap—by asking a small segmentation network to operate inside each aligned proposal.

3. Prerequisites

Faster R-CNN/RPN, semantic vs instance segmentation, convolutions, binary cross-entropy, FPN, ROI Align, IoU, AP, and mask annotation formats.

4. Core Concepts

Concept	Meaning / importance	Example	Interview angle
Instance mask	Per-object binary pixel map	Two cars receive separate masks	Instance vs semantic segmentation
Parallel heads	Box/class and mask predictions share ROI features but solve separate tasks	Box head + small FCN mask head	Why not predict one full-image mask?
ROI Align	Preserves subpixel correspondence crucial for mask boundaries	Bilinear samples at exact coordinates	Why masks exposed ROI Pool's rounding issue
Class-specific masks	Predict K masks and choose the detected class channel	Use "person" mask plane	Why mask head does not use softmax over classes
Multi-task learning	Shared representations are trained by RPN, box, class, and mask signals	Combined five losses	How are heads balanced?

5. Algorithm / Working Process

1. Backbone/FPN extracts multi-scale features.
2. RPN proposes candidate object boxes.
3. ROI Align extracts aligned features for each proposal.
4. Box branch predicts class and refined box.
5. Mask branch uses convolutions and upsampling to predict a low-resolution mask (commonly **28×28**) per class.
6. Training crops/resizes the ground-truth instance mask to the matched ROI and applies per-pixel BCE only to positive ROIs.
7. Inference selects the predicted-class mask, resizes it to the final box, thresholds it, and pastes it into image coordinates.

6. Mathematical Foundation

The full loss is

$$\mathcal{L} = \mathcal{L}_{rpn_cls} + \mathcal{L}_{rpn_box} + \mathcal{L}_{roi_cls} + \mathcal{L}_{roi_box} + \lambda_m \mathcal{L}_{mask}.$$

For a positive ROI with true class k , only channel k contributes:

$$\mathcal{L}_{mask} = -\frac{1}{m^2} \sum_{i,j} [y_{ij} \log \sigma(z_{kij}) + (1 - y_{ij}) \log(1 - \sigma(z_{kij}))].$$

Independent sigmoid masks avoid competition among categories; class competition is handled by the box classification head. Mask IoU is $|M_p \cap M_g| / |M_p \cup M_g|$, and COCO mask AP averages precision across IoU thresholds.

7. Practical Implementation

```
import torch
from torchvision.models.detection import (
    MaskRCNN_ResNet50_FPN_Weights,
    maskrcnn_resnet50_fpn,
)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = maskrcnn_resnet50_fpn(
    weights=MaskRCNN_ResNet50_FPN_Weights.DEFAULT
).to(device).eval()

image = torch.rand(3, 640, 640, device=device)
with torch.no_grad():
    out = model([image])[0]

keep = out["scores"] > 0.7
boxes = out["boxes"][keep]
masks = out["masks"][keep, 0] > 0.5 # [N,H,W] booleans
print(boxes.shape, masks.shape)

# A custom training target also includes one binary mask per object.
target = {
    "boxes": torch.tensor([[40., 50., 200., 300.]], device=device),
    "labels": torch.tensor([1], dtype=torch.int64, device=device),
    "masks": torch.zeros(1, 640, 640, dtype=torch.uint8, device=device),
}
target["masks"][0, 50:300, 40:200] = 1
```

8. Code Explanation

The model returns `boxes`, `labels`, `scores`, and soft `masks` already resized to the input image. Thresholding at 0.5 produces binary masks; tune it on validation data if boundary costs differ. Training masks must correspond one-to-one with boxes/labels and should encode each instance separately.

9. Training / Evaluation

Instance annotations may be polygons, RLE, or bitmaps; rasterization must preserve holes and image alignment. Split by source and audit small/thin objects. Use COCO **mask AP**, box AP, AP by scale, boundary IoU/F-score when edges matter, latency, and memory. Pretraining is usually essential. Tune proposal recall, ROI resolution, mask loss weight, image scale, threshold, and augmentation. Inspect mask overlays rather than trusting aggregate AP.

10. Complexity and Cost

Mask R-CNN includes all Faster R-CNN costs plus a per-positive-ROI convolutional mask head and full-image mask materialization. Memory grows with image size and retained instances. Training is GPU-preferred; high-resolution medical/satellite images often require crops, gradient accumulation, or smaller batches.

11. Common Use Cases

Cell/organ segmentation, robot grasp planning, road-user outlines, crop/weed mapping, damage assessment, foreground extraction, and object-aware editing.

12. Common Mistakes

- Confusing semantic labels with separate instance masks
- Mismatching mask, box, and label order
- Resizing masks with bilinear interpolation and creating invalid labels
- Evaluating box AP but claiming segmentation quality
- Ignoring boundary quality or tiny objects
- Thresholding logits before sigmoid during custom implementation
- Treating crowd regions as ordinary instances

13. Edge Cases / Limitations

Low-resolution ROI masks blur thin structures and precise boundaries. Occluded objects require learning amodal expectations but annotations usually show only visible pixels. Dense scenes stress RPN/NMS. Annotation is expensive. Inference cost and memory rise with instance count, and disconnected mask fragments may appear.

14. Variations

- **Cascade Mask R-CNN**: stronger boxes and masks via cascaded heads; high-accuracy projects.
- **Keypoint R-CNN**: parallel keypoint heatmaps; pose projects and placements.
- **PointRend**: adaptively refines uncertain boundary points; sharp edges, research/projects.
- **YOLACT/SOLO/YOLO-seg**: faster one-stage instance segmentation; deployment.
- **Panoptic FPN**: combines instance “things” and semantic “stuff”; scene understanding.

15. Related Topics

Mask R-CNN vs U-Net: per-instance proposal masks versus dense semantic maps. **Instance vs panoptic segmentation**: panoptic assigns every pixel while distinguishing countable things. **ROI Align vs ROI Pool**:

alignment is critical for pixel outputs. **SAM**: promptable foundation segmentation offers different zero-shot interaction but not native class detection.

16. Interview Questions

1. **What does Mask R-CNN add?** A parallel per-ROI mask head to Faster R-CNN.
2. **Why ROI Align?** It removes coordinate quantization that misaligns feature and pixel boundaries.
3. **Why sigmoid rather than class softmax for masks?** Each selected class channel solves independent foreground/background segmentation.
4. **Is mask loss computed for negative ROIs?** No, only matched positive proposals have target masks.
5. **Instance vs semantic segmentation?** Instance segmentation separates individual objects; semantic segmentation labels pixels by class only.
6. **How is a 28×28 mask used?** Resize it to the detected box and paste/threshold in image coordinates.
7. **Why can box AP be high but mask AP low?** Rectangles can localize objects while boundaries remain inaccurate.
8. **How do you improve thin boundaries?** Increase ROI resolution, use PointRend/boundary loss, stronger features, and accurate labels.
9. **Main data requirement?** A separate aligned mask, box, and class for every instance.
10. **What is panoptic segmentation?** A unified output for instance-level things and semantic stuff.
11. **Why is it slower than Faster R-CNN?** It adds per-ROI mask computation and mask resizing/pasting.

17. Practice Tasks

Visualize masks and boxes; fine-tune on Penn-Fudan; compare nearest-neighbor vs bilinear target resizing; compute mask IoU; add a boundary metric and analyze thin objects.

18. Project Ideas

Project	Function	Stack / dataset	Resume value
Cell Instance Counter	Segments/counts touching cells	PyTorch, OpenCV; BBBC	Biomedical instance analysis
Road Damage Mapper	Produces per-damage masks	Mask R-CNN, FastAPI; custom/RDD	Geospatial deployment
Produce Grading	Segments fruit and estimates size/defects	torchvision, CVAT; custom	Data labeling through serving

19. Quick Revision

- **Idea:** Faster R-CNN plus an aligned per-instance mask branch.
- **Formula:** detector loss + positive-ROI pixelwise BCE.
- **Metrics:** mask AP, boundary F-score, latency.
- **Trap:** semantic masks are not instance annotations.
- **One-liner:** Mask R-CNN turns two-stage detection into instance segmentation through ROI Align and a parallel FCN head.

20. Final Cheat Sheet

Item	Summary
Input / output	Image → boxes, classes, per-instance masks
Steps	FPN → RPN → ROI Align → box and mask heads
Hyperparameters	Proposal count, ROI/mask size, mask threshold, loss weight
Pros / cons	Accurate instance masks / expensive and boundary-limited
Best use	Accuracy-first instance segmentation

Vision Transformer

1. Overview

A **Vision Transformer (ViT)** applies the Transformer encoder to an image represented as a sequence of fixed-size patches. It replaces convolutional spatial processing with global self-attention and learned token mixing. ViTs power classification, detection, segmentation, multimodal encoders, and vision foundation models. They scale strongly with pretraining data and compute.

2. Intuition

Cut an image into square tiles, describe every tile with a vector, add its location, and let each tile compare itself with every other tile. A wheel patch can directly attend to another wheel and the vehicle body even when far apart. A special summary token collects evidence for classification.

3. Prerequisites

Linear algebra, CNN basics, embeddings, attention, softmax, residual connections, LayerNorm, MLPs, positional encodings, cross-entropy, and transfer learning.

4. Core Concepts

Concept	Meaning / importance	Example	Interview angle
Patch embedding	Flatten/project each $P \times P \times C$ patch into dimension D ; equivalent to stride- P convolution	224×224 , $P=16$ gives 196 tokens	How does patch size affect cost?
Position embedding	Adds spatial-order information absent from attention	Learn one vector per grid position	What happens at a new resolution?
[CLS] token	Learned token used as global representation	Final CLS → classifier	CLS vs global average pooling
Multi-head self-attention	Learns multiple token relationships	Heads focus on shape, texture, context	Why multiple heads?
Pre-norm block	LayerNorm before attention/MLP plus residuals	$x += \text{MSA}(\text{LN}(x))$	Why residuals and LayerNorm?

Concept	Meaning / importance	Example	Interview angle
Inductive bias	ViT has less built-in locality/translation equivariance than CNNs	Needs data/augmentation	CNN vs ViT data efficiency

5. Algorithm / Working Process

1. **Input:** image $[B, C, H, W]$ with dimensions divisible by patch size P .
2. Patchify into $N=HW/P^2$ patches and linearly project each to D dimensions.
3. Prepend $[CLS]$ (for classic ViT) and add positional embeddings.
4. Repeat L encoder blocks: LayerNorm $\rightarrow$ multi-head attention $\rightarrow$ residual $\rightarrow$ LayerNorm $\rightarrow$ MLP $\rightarrow$ residual.
5. Normalize the final sequence; pass CLS or pooled patch tokens to a classification head.
6. Train with cross-entropy, usually using large-scale supervised or self-supervised pretraining.
7. At inference, use deterministic preprocessing and one forward pass; dense tasks reshape patch tokens back to a 2-D grid.

6. Mathematical Foundation

Patch embeddings are

$$Z_0 = [x_{cls}; x_p^1 E; \dots; x_p^N E] + E_{pos}.$$

For each head,

$$Q = ZW_Q, K = ZW_K, V = ZW_V, \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V.$$

Heads are concatenated and projected. An encoder block computes

$$Z'_l = Z_{l-1} + \text{MSA}(\text{LN}(Z_{l-1})), \quad Z_l = Z'_l + \text{MLP}(\text{LN}(Z'_l)).$$

Classification minimizes $-\sum_k y_k \log p_k$. Attention memory/time is $O(N^2)$; halving patch size makes four times as many tokens and roughly sixteen times the attention matrix work.

7. Practical Implementation

```
import torch
from torch import nn

class TinyViT(nn.Module):
    def __init__(self, image_size=32, patch=4, dim=128, depth=4,
                 heads=4, classes=10):
        super().__init__()
        n = (image_size // patch) ** 2
        self.patch_embed = nn.Conv2d(3, dim, kernel_size=patch, stride=patch)
        self.cls = nn.Parameter(torch.zeros(1, 1, dim))
        self.pos = nn.Parameter(torch.randn(1, n + 1, dim) * 0.02)
        layer = nn.TransformerEncoderLayer(
            dim, heads, dim_feedforward=4 * dim,
```

```

        dropout=0.1, batch_first=True, norm_first=True,
    )
    self.encoder = nn.TransformerEncoder(layer, depth)
    self.norm = nn.LayerNorm(dim)
    self.head = nn.Linear(dim, classes)

    def forward(self, x):
        x = self.patch_embed(x).flatten(2).transpose(1, 2)
        cls = self.cls.expand(x.size(0), -1, -1)
        x = torch.cat([cls, x], dim=1) + self.pos
        return self.head(self.norm(self.encoder(x))[:, 0])

model = TinyViT()
images = torch.randn(8, 3, 32, 32)
labels = torch.randint(0, 10, (8,))
loss = nn.CrossEntropyLoss()(model(images), labels)
loss.backward()
print(float(loss), model(images).shape)

```

8. Code Explanation

The stride-4 convolution performs patch extraction and projection in one operation.

`flatten(2).transpose(1,2)` converts the spatial grid to $[B, N, D]$. CLS and position parameters are optimized with the model. PyTorch's encoder supplies pre-norm attention, MLPs, residuals, and dropout. The head reads only the final CLS vector.

9. Training / Evaluation

Use exact pretrained normalization and resolution when fine-tuning. Strong augmentation (RandAugment, Mixup, CutMix), label smoothing, AdamW, warmup, cosine decay, stochastic depth, and weight decay are common. For small data, prefer pretrained weights, smaller ViT/hybrid CNN, or DeiT-style distillation. Report accuracy/top-5, macro-F1 for imbalance, calibration, latency, throughput, parameters, and FLOPs. Interpolate 2-D positional embeddings when changing the patch grid.

10. Complexity and Cost

With N tokens and width D , attention costs roughly $O(N^2 D)$, while projections/MLP cost $O(ND^2)$. High resolution is expensive because $N=HW/P^2$. ViTs parallelize well on GPUs/TPUs but can be inefficient on CPUs and small batches. Patch size, depth, width, head count, and precision dominate memory and latency.

11. Common Use Cases

Image classification, medical/satellite recognition, backbone for detection/segmentation, self-supervised representation learning, video transformers, and visual encoders in CLIP/VLMs.

12. Common Mistakes

- Training a large ViT from scratch on a tiny dataset
- Forgetting position-embedding interpolation at a new resolution
- Miscounting tokens or mixing $[B, N, D]$ and $[N, B, D]$

- Using normalization different from the pretrained checkpoint
- Claiming attention maps are faithful explanations
- Ignoring quadratic resolution cost
- Comparing with a CNN under unequal pretraining/augmentation

13. Edge Cases / Limitations

Vanilla ViT can be data-hungry and costly at high resolution. Fixed patches may lose small/local detail and create boundary artifacts. Absolute positions complicate variable resolution. Global attention consumes quadratic memory. Robustness and calibration can degrade under distribution shift; attention weights alone do not prove causal importance.

14. Variations

- **DeiT**: data-efficient training and distillation; placement-important.
- **Swin Transformer**: shifted local windows and hierarchy; dense vision/projects.
- **MAE/DINO**: masked or self-distilled pretraining; foundation/research.
- **Hybrid CNN-ViT**: convolutional stem adds locality; smaller data.
- **MobileViT/EfficientViT**: hardware-aware efficient blocks; deployment.

15. Related Topics

CNN vs ViT: strong local equivariant bias versus flexible global token interactions. **ViT vs Swin**: global flat attention versus hierarchical window attention. **ViT and CLIP**: a ViT often encodes images into the shared contrastive space. **MAE vs supervised pretraining**: reconstruction/self-supervision versus label-based objectives.

16. Interview Questions

1. **How does ViT convert an image to a sequence?** Split into patches, flatten/project them, and add positions.
2. **Why positional embeddings?** Self-attention alone is permutation equivariant and does not know spatial order.
3. **Purpose of CLS?** It is a learned global token whose final representation feeds the classifier.
4. **Why divide by $\sqrt{d_k}$?** To keep dot-product variance controlled and avoid saturated softmax.
5. **Main ViT complexity?** Quadratic attention in the number of patches.
6. **Effect of smaller patches?** Better spatial detail but many more tokens and much higher cost.
7. **Why is ViT data-hungry?** It lacks some local/translation inductive biases built into CNNs.
8. **How handle a larger fine-tuning image?** Interpolate grid positional embeddings and keep the CLS embedding.
9. **LayerNorm vs BatchNorm?** LayerNorm normalizes features per token/sample and is independent of batch statistics.
10. **How does Swin reduce cost?** It attends within local windows and shifts windows to exchange information.
11. **Are attention weights explanations?** Not necessarily; they show internal weighting, not guaranteed causal attribution.

17. Practice Tasks

Implement patchification two ways; train TinyViT on CIFAR-10; compare patch sizes; visualize positional similarity/attention cautiously; debug a positional-size mismatch when switching resolutions.

18. Project Ideas

Project	Function	Stack / dataset	Resume value
Plant Disease ViT	Classifies leaf disease with shift testing	PyTorch/timm; PlantVillage	Transfer learning and robustness
Satellite Land Classifier	Predicts land-use categories	ViT, Rasterio; EuroSAT	Geospatial pipeline
CNN-vs-ViT Benchmark	Controlled accuracy/cost comparison	PyTorch, MLflow; CIFAR/ImageNet subset	Experimental rigor

19. Quick Revision

- **Idea:** image patches become Transformer tokens.
- **Formula:** $\text{softmax}(QK^T/\sqrt{d})V$.
- **Use:** scalable pretrained vision representations.
- **Metrics:** task metric plus FLOPs/latency.
- **Trap:** quadratic token cost and small-data training.
- **One-liner:** ViT trades convolutional locality for global self-attention over patch tokens.

20. Final Cheat Sheet

Item	Summary
Input / output	Image patches → class/visual features
Steps	Patch embed + position → encoder blocks → pool/head
Hyperparameters	Patch, width, depth, heads, LR, weight decay, drop path
Pros / cons	Scales and transfers well / data and resolution cost
Best use	Pretrained classification and vision backbones

CLIP

1. Overview

CLIP (Contrastive Language–Image Pre-training) jointly trains an image encoder and text encoder on paired image-caption data. Matching images and texts are pulled together in a shared embedding space while mismatched pairs are pushed apart. This enables zero-shot classification, cross-modal retrieval, semantic search, transfer learning, and conditioning/evaluation in generative systems.

2. Intuition

In a batch of captioned images, treat the true caption as the correct answer for each image and every other caption as a negative. Also perform the reverse task: find the correct image for each caption. After many pairs, “a red sports car” lands near relevant images even if “sports car” was never a fixed training class.

3. Prerequisites

CNN/ViT image encoders, Transformer text encoders, tokenization, embeddings, cosine similarity, softmax cross-entropy, contrastive learning, batching, and transfer learning.

4. Core Concepts

Concept	Meaning / importance	Example	Interview angle
Dual encoder	Image/text processed independently; embeddings can be indexed	Precompute 1M image vectors	Why retrieval is efficient
Shared normalized space	Unit vectors make dot product cosine similarity	Image dog near text “a dog”	Why L2 normalization?
In-batch negatives	Other pairs supply negatives without a separate sampler	Batch size 256 gives 255 negatives	Why large batches help
Symmetric objective	Image→text and text→image cross-entropies	Two retrieval directions	Write InfoNCE loss
Temperature/logit scale	Controls softmax sharpness	Lower temperature emphasizes hard alternatives	Why learn temperature?
Prompting	Class names become natural-language descriptions	“a photo of a {class}”	Why prompt ensembles help

5. Algorithm / Working Process

1. Sample B matched (image, text) pairs.
2. Encode images with a CNN/ViT and text with a Transformer.
3. Project both to dimension d and L2-normalize.
4. Form all $B \times B$ scaled cosine similarities.
5. Use diagonal pairs as positives and off-diagonal pairs as negatives.
6. Minimize image-to-text plus text-to-image cross-entropy.
7. For zero-shot classification, encode prompted class names, compare an image vector to them, and choose the highest similarity.

6. Mathematical Foundation

Let normalized embeddings be v_i and t_j , and temperature τ :

$$s_{ij} = \frac{v_i^T t_j}{\tau}.$$

The symmetric contrastive loss is

$$\mathcal{L} = \frac{1}{2B} \sum_i \left[-\log \frac{e^{s_{ii}}}{\sum_j e^{s_{ij}}} - \log \frac{e^{s_{ii}}}{\sum_j e^{s_{ji}}} \right].$$

For zero-shot class c , encode one or more templates and average normalized text features. Predict

$$p(c \mid x) = \text{softmax}_c(v(x)^T t(c)/\tau).$$

This probability is relative to the supplied candidate labels and is not automatically calibrated for real-world prevalence.

7. Practical Implementation

```
# pip install transformers pillow
import torch
from PIL import Image
from transformers import CLIPModel, CLIPProcessor

name = "openai/clip-vit-base-patch32"
model = CLIPModel.from_pretrained(name).eval()
processor = CLIPProcessor.from_pretrained(name)

image = Image.open("test.jpg").convert("RGB")
labels = ["a photo of a cat", "a photo of a dog", "a photo of a bicycle"]
inputs = processor(text=labels, images=image, return_tensors="pt", padding=True)

with torch.no_grad():
    outputs = model(**inputs)
    probabilities = outputs.logits_per_image.softmax(dim=-1)[0]
print(dict(zip(labels, probabilities.tolist())))
```

Minimal symmetric loss:

```
import torch.nn.functional as F

def clip_loss(image_features, text_features, temperature=0.07):
    image_features = F.normalize(image_features, dim=-1)
    text_features = F.normalize(text_features, dim=-1)
    logits = image_features @ text_features.T / temperature
    target = torch.arange(logits.size(0), device=logits.device)
    return (F.cross_entropy(logits, target) +
            F.cross_entropy(logits.T, target)) / 2
```

8. Code Explanation

The processor applies the checkpoint's exact image resizing/normalization and text tokenization.

`logits_per_image` contains scaled pairwise similarities between one image and each prompt. The custom loss makes the diagonal the correct class in both directions; therefore pair ordering must remain aligned.

9. Training / Evaluation

Pretraining requires large, diverse, clean-enough paired data and substantial distributed compute. Fine-tuning can freeze encoders, learn a linear probe, use adapters, or optimize both towers carefully. Evaluate image→text/text→image Recall@K, median rank, zero-shot/linear-probe accuracy, subgroup results, calibration, latency, and embedding drift. Ensure split deduplication across near-identical images/captions. Hard-negative mining helps but false negatives must be handled.

10. Complexity and Cost

Encoding cost is the sum of the two encoders. Similarity training constructs a $B \times B$ matrix: $O(B^2d)$ compute/memory locally, often with embeddings gathered across devices. Retrieval is efficient because database embeddings are precomputed; exact search is $O(Nd)$, while approximate nearest-neighbor indexes trade recall for speed.

11. Common Use Cases

Zero-shot classification, image/text search, content moderation assistance, dataset labeling, duplicate discovery, visual recommendation, multimodal RAG retrieval, and text conditioning or scoring for image generation.

12. Common Mistakes

- Using bare class words instead of suitable prompt templates
- Forgetting L2 normalization or learned logit scale
- Breaking image-caption alignment when shuffling
- Treating every off-diagonal pair as truly negative
- Interpreting candidate-softmax scores as calibrated confidence
- Ignoring societal/dataset bias and unsafe failure modes
- Fine-tuning one tower so aggressively that the shared space collapses

13. Edge Cases / Limitations

CLIP can prefer textual shortcuts, struggle with counting, spatial relations, fine-grained categories, OCR, and unseen visual domains. Web data transfers bias and harmful associations. Dual encoders interact only through final similarity, limiting deep compositional reasoning. Zero-shot results depend heavily on prompt and candidate set.

14. Variations

- **OpenCLIP**: open training recipes/checkpoints; practical projects.
- **SigLIP**: sigmoid pairwise objective rather than batch softmax; modern placement/research.
- **ALIGN**: large noisy image-text pretraining; conceptual importance.
- **LiT**: locks a pretrained image tower while tuning text; efficient transfer.
- **Domain CLIP (BioCLIP/MedCLIP)**: specialized paired data; domain projects/research.

15. Related Topics

CLIP vs cross-encoder: fast independently indexable representations versus deeper but expensive joint interaction. **CLIP vs supervised classifier:** open-vocabulary prompted labels versus fixed learned head. **CLIP and Stable Diffusion:** the text encoder supplies conditioning, while the diffusion model generates latents. **CLIP and vector databases:** CLIP vectors enable semantic multimodal retrieval.

16. Interview Questions

1. **What is CLIP's objective?** Symmetric contrastive matching of true image-text pairs against in-batch alternatives.
2. **Why dual encoders?** They allow independent precomputation and scalable retrieval.
3. **Why normalize embeddings?** Dot product becomes cosine similarity and vector norm cannot trivially dominate.
4. **Why large batches?** They supply many diverse in-batch negatives.
5. **What does temperature do?** It controls similarity-softmax sharpness and gradient emphasis.
6. **How is zero-shot classification performed?** Encode class prompts and choose the text embedding most similar to the image.
7. **What is a false negative?** An off-diagonal pair that is semantically also correct, creating a misleading repulsive signal.
8. **How evaluate retrieval?** Recall@K, rank statistics, and subgroup/domain analysis.
9. **CLIP vs captioning model?** CLIP scores compatibility; it does not autoregressively generate captions.
10. **Why prompt ensembling?** Multiple templates reduce sensitivity to wording/context mismatch.
11. **Biggest production concern?** Domain/bias evaluation and score calibration for the actual candidate distribution.

17. Practice Tasks

Implement the symmetric loss; build a small image search index; compare prompt templates; find false negatives in a batch; fine-tune a linear probe and compare with zero-shot accuracy.

18. Project Ideas

Project	Function	Stack / dataset	Resume value
Multimodal Search	Searches an image catalog using text or images	CLIP, FAISS, FastAPI; Unsplash subset	Retrieval systems
Zero-shot QA Auditor	Evaluates labels/prompts across subgroups	PyTorch, CLIP; FairFace/domain data	Responsible evaluation
Visual RAG	Retrieves figures/images for a user query	CLIP, vector DB, VLM	Multimodal AI pipeline

19. Quick Revision

- **Idea:** align images and text through symmetric contrastive learning.
- **Formula:** diagonal InfoNCE over a similarity matrix.
- **Use:** open-vocabulary classification and retrieval.
- **Metrics:** Recall@K, zero-shot accuracy, calibration.
- **Trap:** prompt dependence and false negatives.

- **One-liner:** CLIP learns a shared normalized space where matched images and captions have high cosine similarity.

20. Final Cheat Sheet

Item	Summary
Input / output	Image or text → shared embedding
Steps	Two encoders → normalize → scaled similarity → symmetric CE
Hyperparameters	Batch, temperature, embedding dim, prompts, LR
Pros / cons	Open-vocabulary, indexable / bias, weak fine reasoning
Best use	Cross-modal retrieval and zero-shot transfer

DQN

1. Overview

A **Deep Q-Network (DQN)** uses a neural network to approximate the optimal action-value function $Q^*(s, a)$ for discrete-action reinforcement learning. Its key stabilizers are an experience replay buffer, which breaks temporal correlations and reuses transitions, and a target network, which makes bootstrap targets change slowly. DQN established strong Atari results from pixels and remains a foundational interview algorithm for value-based RL.

2. Intuition

The agent maintains a scoreboard: for the current situation, what long-term return should each possible action produce? It sometimes explores, otherwise chooses the highest score. After observing reward and the next situation, it moves the chosen action's score toward "reward plus the best future score." Old experiences are shuffled before practice, and a slowly updated copy supplies stable answer targets.

3. Prerequisites

Markov decision processes (MDPs), states/actions/rewards, return and discount factor, Bellman equations, Q-learning, epsilon-greedy exploration, temporal-difference learning, neural networks, replay buffers, and gradient clipping.

4. Core Concepts

Concept	Meaning / why it matters	Example	Interview angle
Q-network	Outputs one value per discrete action	Atari state → 18 Q-values	Why not continuous actions?
TD target/error	Bootstrapped learning signal	$r + \gamma \max_a Q_{\text{target}}(s')$	Semi-gradient target

Concept	Meaning / why it matters	Example	Interview angle
Replay buffer	Stores and randomly samples transitions	Last 100k steps	Why decorrelate data?
Target network	Delayed copy used in targets	Sync every 1,000 updates	Why stabilization?
Epsilon-greedy	Random action with probability epsilon	Decay 1.0 to 0.05	Exploration limits
Terminal masking	Future value is zero after true termination	Game over vs time limit	Terminated vs truncated

5. Algorithm / Working Process

1. Initialize online network Q_θ , target network $Q_{\bar{\theta}} = Q_\theta$, and replay buffer.
2. Observe state and choose a random action with probability ϵ ; otherwise choose $\arg \max_a Q_\theta(s, a)$.
3. Execute action, observe reward, next state, termination, and store transition.
4. After warm-up, sample a random mini-batch from replay.
5. Compute target $y = r + \gamma(1 - d) \max_{a'} Q_{\bar{\theta}}(s', a')$.
6. Regress the online value for the selected action toward y using Huber/MSE loss.
7. Periodically hard-copy or softly update target parameters.
8. Repeat; evaluation uses greedy actions and no learning.

6. Mathematical Foundation

The optimal Bellman equation is

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a') \mid s, a].$$

For sampled transition (s, a, r, s', d) :

$$y = r + \gamma(1 - d) \max_{a'} Q_{\bar{\theta}}(s', a'), \quad \delta = y - Q_\theta(s, a).$$

The Huber loss is quadratic for small error and linear for large error:

$$L_\kappa(\delta) = \begin{cases} \frac{1}{2}\delta^2, & |\delta| \leq \kappa \\ \kappa(|\delta| - \frac{1}{2}\kappa), & \text{otherwise.} \end{cases}$$

DQN is off-policy: replay data may come from older epsilon-greedy policies while learning a greedy target. The “deadly triad”—function approximation, bootstrapping, and off-policy learning—explains potential divergence.

7. Practical Implementation

```
from collections import deque
import torch
from torch import nn
import torch.nn.functional as F
```

```

class QNet(nn.Module):
    def __init__(self, obs_dim, actions):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(obs_dim, 128), nn.ReLU(),
                                   nn.Linear(128, 128), nn.ReLU(),
                                   nn.Linear(128, actions))

    def forward(self, x):
        return self.net(x)

online, target = QNet(4, 2), QNet(4, 2)
target.load_state_dict(online.state_dict())
optimizer = torch.optim.Adam(online.parameters(), lr=1e-3)
replay = deque(maxlen=50_000)

def train_step(batch, gamma=0.99):
    states, actions, rewards, next_states, terminated = map(torch.stack,
zip(*batch))
    q = online(states).gather(1, actions.long().unsqueeze(1)).squeeze(1)
    with torch.no_grad():
        next_q = target(next_states).max(1).values
        y = rewards + gamma * (~terminated.bool()).float() * next_q
    loss = F.smooth_l1_loss(q, y)
    optimizer.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(online.parameters(), 10.0)
    optimizer.step()
    return loss.item()

# Periodically: target.load_state_dict(online.state_dict())

```

8. Code Explanation

The final layer emits all discrete action values. `gather` selects only the value of the action actually taken. Target computation is under `no_grad`, preventing optimization through the bootstrap branch. The mask removes future value only for true terminal states. `smooth_l1_loss` and gradient clipping limit destructive updates.

9. Training / Evaluation

Normalize observations and clip/scale rewards only with a documented reason. Fill the replay buffer before updates; control training-to-environment step ratio. Evaluate periodically with fixed seeds and greedy policy across many episodes; report mean/median return, confidence intervals, success rate, sample efficiency, and wall-clock time. Separate training and evaluation environments. Tune LR, gamma, buffer size, batch, warm-up, update frequency, target interval/Polyak factor, epsilon schedule, and network capacity.

10. Complexity and Cost

Each action selection costs one network forward pass; each update costs one forward/backward mini-batch plus a target forward pass. Replay memory is $O(N \cdot \text{transition size})$, particularly large for images; frame compression/stacking is essential. Output and max cost grow linearly with the number of discrete actions.

DQN is usually more sample-efficient than on-policy methods due to replay but can be compute-heavy with frequent updates.

11. Common Use Cases

Atari/control games, discrete resource allocation, scheduling, simple robot action sets, educational RL, and simulated decision systems with a manageable discrete action space.

12. Common Mistakes

- Forgetting to detach/no-grad the TD target
- Using the online network for a rapidly moving target without stabilization
- Masking time-limit truncations as true terminal states
- Evaluating with epsilon exploration enabled
- Starting updates before replay has diverse data
- Feeding uint8 images without correct scaling/frame history
- Claiming learning from one lucky evaluation seed
- Applying vanilla DQN directly to continuous actions

13. Edge Cases / Limitations

DQN is unsuitable for unbounded/large continuous actions because it requires max over actions. It may overestimate Q-values, learn slowly with sparse rewards, forget under nonstationarity, and exploit simulator bugs. Replay is hard with recurrent state unless sequences are stored. Offline DQN can extrapolate optimistically to unsupported actions.

14. Variations

- **Double DQN:** online network selects action, target evaluates it; reduces overestimation, placement-essential.
- **Dueling DQN:** separates state value and action advantage; useful when actions have similar value.
- **Prioritized replay:** samples high-TD-error transitions with importance correction; projects/interviews.
- **NoisyNet:** parameter noise for learned exploration.
- **Distributional DQN (C51/QR-DQN):** models return distribution; advanced/research.
- **Rainbow:** combines major improvements; strong benchmark.

15. Related Topics

DQN vs tabular Q-learning: neural generalization over states versus a lookup table. **DQN vs PPO:** off-policy value learning with replay/discrete actions versus on-policy policy-gradient optimization. **DQN vs actor-critic:** one value network implicitly defines greedy policy versus separate policy and critic. **Double DQN vs target network:** target network stabilizes; Double DQN decouples selection/evaluation bias.

16. Interview Questions

1. **Why experience replay?** It reuses data and reduces correlation/nonstationarity in consecutive transitions.
2. **Why a target network?** It slows target movement and stabilizes bootstrapped regression.
3. **Is DQN on-policy?** No, it learns a greedy target from replay generated by older behavior policies.

4. **How is the action value selected in the loss?** Gather the output corresponding to the executed action.
5. **Why does vanilla DQN overestimate?** The same noisy estimator selects and evaluates the maximum action.
6. **How does Double DQN help?** Online network chooses argmax; target network evaluates that chosen action.
7. **Why not continuous actions?** Computing an exact max over an infinite action space is impractical.
8. **What is the deadly triad?** Off-policy learning, bootstrapping, and function approximation together can diverge.
9. **Terminal vs truncated?** Terminal ends the MDP and zeroes bootstrap; truncation may still bootstrap the continuing value.
10. **Why Huber loss?** It is less sensitive than MSE to large noisy TD errors.
11. **How do you evaluate RL reliably?** Multiple fixed/unseen seeds, confidence intervals, learning curves, success and wall-clock/sample efficiency.

17. Practice Tasks

Implement epsilon-greedy and replay; solve CartPole; compare online-only vs target network; implement Double DQN; debug a run where truncations are incorrectly masked; plot Q scale and TD errors.

18. Project Ideas

Project	Function	Stack / environment	Resume value
CartPole DQN Lab	Compares DQN stabilizers ablation-by-ablation	PyTorch, Gymnasium	Correct RL experimentation
Warehouse Scheduler	Selects discrete dispatch actions	custom Gymnasium env	Environment/reward design
Atari Rainbow Study	Adds Double/dueling/prioritized components	PyTorch, ALE	Advanced value-based RL

19. Quick Revision

- **Idea:** neural Q-learning stabilized by replay and a target network.
- **Formula:** $y = r + \gamma \max_a Q_{\text{target}}(s', a')$.
- **Use:** discrete actions with simulation/data interaction.
- **Metrics:** return, success, sample/wall-clock efficiency.
- **Trap:** target gradients, termination masking, single-seed claims.
- **One-liner:** DQN learns discrete action values from replayed TD targets supplied by a slowly changing target network.

20. Final Cheat Sheet

Item	Summary
Input / output	State → Q-value per discrete action
Steps	Epsilon action → store → sample → TD loss → target sync

Item	Summary
Hyperparameters	LR, gamma, epsilon, buffer, batch, target interval
Pros / cons	Reuses data, simple policy / discrete only, unstable/sparse reward
Best use	Discrete-action simulated environments

PPO

1. Overview

Proximal Policy Optimization (PPO) is an on-policy actor-critic algorithm that improves a stochastic policy while limiting how much action probabilities change per update. Its clipped surrogate objective is simpler than trust-region optimization and robust enough to become a standard baseline for continuous/discrete control and RL fine-tuning. PPO collects fresh rollouts, estimates advantages, and performs several mini-batch epochs before discarding the data.

2. Intuition

If an action worked better than expected, make it more likely—but not so much that one noisy batch rewrites the policy. PPO compares the new probability of each sampled action with the old probability. Once the ratio moves outside a safe band, clipping removes the incentive to push farther in that direction.

3. Prerequisites

Policy gradients, stochastic policies/distributions, log-probability, importance sampling, actor-critic, value functions, TD error, advantage estimation, entropy, and on-policy rollouts.

4. Core Concepts

Concept	Meaning / why it matters	Example	Interview angle
Probability ratio	$\pi_{\text{new}}(a$	$s)/\pi_{\text{old}}(a$	$s)$ measures policy change on sampled action
Clipped objective	Caps incentive outside $[1-\text{eps}, 1+\text{eps}]$	Epsilon .2	Explain min and advantage sign
Critic	Predicts state value for baseline/bootstrap	$V(s)$	Reduces gradient variance
GAE	Bias-variance-controlled advantage estimator	Lambda .95	Roles of gamma and lambda
Entropy bonus	Encourages non-collapsed exploration	Categorical/Gaussian entropy	Exploration vs convergence
Rollout epochs	Reuse one on-policy batch several times	4–10 epochs	Why too many epochs hurt?

5. Algorithm / Working Process

1. Run policy π_{old} in parallel environments for T steps, storing states, actions, rewards, terminations, old log-probabilities, and values.
2. Bootstrap the final value when the trajectory is not truly terminal.
3. Compute TD residuals, generalized advantage estimates (GAE), and returns.
4. Normalize advantages over the rollout batch.
5. For several shuffled mini-batch epochs, recompute new log-probabilities/values.
6. Optimize clipped policy objective, value loss, and entropy bonus; clip gradients.
7. Stop epochs early if approximate KL is too high (common safeguard).
8. Discard rollout and collect fresh data with the updated policy.

6. Mathematical Foundation

Probability ratio and clipped objective:

$$r_t(\theta) = \exp[\log \pi_\theta(a_t | s_t) - \log \pi_{old}(a_t | s_t)],$$

$$L^{CLIP} = \mathbb{E}_t [\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)].$$

GAE uses

$$\delta_t = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t), \quad A_t^{GAE} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}.$$

A minimized combined loss is

$$\mathcal{L} = -L^{CLIP} + c_v \mathbb{E}[(V_\theta - R)^2] - c_e \mathbb{E}[\mathcal{H}(\pi_\theta)].$$

Clipping is not a hard KL constraint; KL monitoring remains useful.

7. Practical Implementation

```
import torch
import torch.nn.functional as F

def ppo_loss(actor, critic, states, actions, old_logp,
             advantages, returns, clip_eps=0.2,
             value_coef=0.5, entropy_coef=0.01):
    dist = actor(states) # returns torch.distributions.Distribution
    new_logp = dist.log_prob(actions)
    # Sum action dimensions for diagonal Gaussian policies.
    if new_logp.ndim > 1:
        new_logp = new_logp.sum(-1)
    entropy = dist.entropy()
    if entropy.ndim > 1:
        entropy = entropy.sum(-1)

    ratio = (new_logp - old_logp).exp()
    advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)
    unclipped = ratio * advantages
```

```

clipped = ratio.clamp(1 - clip_eps, 1 + clip_eps) * advantages
policy_loss = -torch.min(unclipped, clipped).mean()

values = critic(states).squeeze(-1)
value_loss = F.mse_loss(values, returns)
total = policy_loss + value_coef * value_loss - entropy_coef * entropy.mean()
stats = {"policy": policy_loss.item(), "value": value_loss.item(),
        "entropy": entropy.mean().item(),
        "approx_kl": (old_logp - new_logp).mean().item(),
        "clip_fraction": ((ratio - 1).abs() >
clip_eps).float().mean().item()}
return total, stats

```

8. Code Explanation

Log-probability differences compute a stable importance ratio. Continuous-action distributions return one log-prob per dimension, so they must be summed to obtain the joint log-probability. `min` makes the pessimistic surrogate for both positive and negative advantages. Normalizing advantages improves optimization scale. KL and clip fraction diagnose overly large updates.

9. Training / Evaluation

Use vectorized environments and normalize observations; reward normalization/clipping changes the problem and should be documented. Separate true termination from time-limit truncation when bootstrapping. Report return distributions and success over many seeds, sample count, wall-clock, policy entropy, explained variance, approximate KL, clip fraction, and evaluation without training noise when appropriate. Tune LR, rollout horizon, number of environments, gamma, GAE lambda, clip epsilon, epochs, mini-batch, entropy/value coefficients, max gradient norm, and target KL.

10. Complexity and Cost

PPO needs environment interaction for each fresh rollout and multiple forward/backward passes over it. Memory is $O(TN)$ for N environments. It is less sample-efficient than off-policy methods because old batches are discarded, but vectorization offers high throughput and stable GPU utilization. For large language models, storing logits, values, activations, and reference-policy calculations makes PPO extremely expensive.

11. Common Use Cases

Robotics/control simulation, locomotion, games, scheduling, continuous and discrete policies, curriculum learning, and historically RLHF policy optimization for language models.

12. Common Mistakes

- Recomputing "old" log-probabilities after updating the policy
- Failing to sum log-probs across continuous action dimensions
- Reusing rollout data for too many epochs and violating near-on-policy assumptions
- Wrong GAE boundaries across episodes or vector environments
- Treating truncations as terminals
- Optimizing the wrong sign of the clipped objective

- Omitting action squashing/log-prob correction for bounded continuous actions
- Reporting training stochastic return as deterministic evaluation

13. Edge Cases / Limitations

PPO remains sensitive to reward design, scale, horizon, seeds, and environment bugs. Clipping can stop useful learning or fail to prevent large global KL shifts. On-policy data is expensive for real-world systems. Sparse/long-horizon credit assignment and unsafe exploration remain difficult. Policies may exploit reward loopholes.

14. Variations

- **PPO-Clip**: standard clipped surrogate; placement-essential.
- **PPO-Penalty**: adaptive KL penalty rather than ratio clipping.
- **Recurrent PPO**: RNN state for partial observability; sequence batching matters.
- **Multi-agent PPO/MAPPO**: centralized critic/decentralized actors; research/projects.
- **RLHF PPO**: adds reward model, reference KL, token-level values; LLM context.

15. Related Topics

PPO vs TRPO: first-order clipping approximation versus constrained natural-gradient trust region. **PPO vs DQN**: on-policy stochastic actor-critic for continuous/discrete actions versus replay-based discrete value learning. **PPO vs A2C**: multiple epochs with clipped ratio versus direct advantage policy-gradient updates. **PPO vs SAC**: on-policy robust baseline versus off-policy entropy-regularized continuous control.

16. Interview Questions

1. **Why is PPO “proximal”?** It discourages policy updates that move action probabilities too far from the rollout policy.
2. **What is the probability ratio?** New divided by old probability of the sampled action, computed from log-probs.
3. **Explain clipping for positive advantage.** Once ratio exceeds $1+\epsilon$, extra probability increase yields no larger surrogate reward.
4. **And negative advantage?** The pessimistic minimum prevents decreasing a bad action’s probability beyond the safe ratio from improving the objective unchecked.
5. **Why a critic?** It supplies a baseline/bootstrap that reduces policy-gradient variance.
6. **What is GAE?** Exponentially weighted TD residuals controlled by gamma and lambda.
7. **Is clipping a guaranteed trust region?** No; monitor KL and optionally early-stop.
8. **Why is PPO on-policy?** Ratios remain reliable only near the policy that generated the rollout; old data is soon discarded.
9. **Why entropy bonus?** It resists premature policy collapse and encourages exploration.
10. **What diagnostics matter?** Approximate KL, clip fraction, entropy, value loss/explained variance, return, and gradient norm.
11. **How handle bounded continuous actions?** Often tanh-transform Gaussian samples and include the change-of-variables correction in log-probability.

17. Practice Tasks

Derive the clipping cases; implement GAE; train PPO on CartPole/Pendulum; ablate advantage normalization; debug wrong continuous-action log-probs; sweep epochs and plot KL/clip fraction.

18. Project Ideas

Project	Function	Stack / environment	Resume value
PPO Control Dashboard	Trains/evaluates continuous control with diagnostics	PyTorch, Gymnasium, MLflow	Reproducible RL engineering
Energy Controller	Controls battery/HVAC in simulation	custom Gymnasium env	Reward/constraint design
Multi-agent Traffic Lights	Coordinates intersections	MAPPO/PPO, SUMO	Multi-agent systems

19. Quick Revision

- **Idea:** reuse fresh rollouts for conservative clipped policy updates.
- **Formula:** $\min(rA, \text{clip}(r, 1-\epsilon, 1+\epsilon)A)$ plus value/entropy terms.
- **Use:** robust continuous or discrete on-policy control.
- **Metrics:** return, success, KL, entropy, clip fraction, sample cost.
- **Trap:** stale old log-probs and incorrect GAE termination.
- **One-liner:** PPO improves an actor with GAE while clipping likelihood-ratio incentives to avoid destructive policy jumps.

20. Final Cheat Sheet

Item	Summary
Input / output	State $\rightarrow$ action distribution and value
Steps	Rollout $\rightarrow$ GAE/returns $\rightarrow$ clipped mini-batch epochs $\rightarrow$ discard
Hyperparameters	LR, horizon, gamma, lambda, epsilon, epochs, entropy
Pros / cons	Stable, general / on-policy sample cost, reward sensitivity
Best use	Simulated control and strong policy-gradient baseline

Actor-Critic

1. Overview

Actor-Critic is a family of reinforcement-learning methods with two roles: an **actor** parameterizes the policy $\pi_{\theta}(a | s)$, and a **critic** estimates value $V_w(s)$ or $Q_w(s, a)$. The critic converts rewards into a lower-variance learning signal (TD error/advantage) for the actor. A2C/A3C, PPO, DDPG, TD3, and SAC are actor-critic algorithms.

2. Intuition

The actor is a player choosing moves. The critic is a coach who predicts how promising the current situation is and says whether the latest result was better or worse than expected. The actor repeats unexpectedly good actions and avoids unexpectedly bad ones; the critic continually improves its expectations.

3. Prerequisites

MDPs, returns, state/action values, Bellman expectation equations, policy-gradient theorem, log-derivative trick, TD learning, baselines, stochastic/continuous distributions, and on-policy vs off-policy learning.

4. Core Concepts

Concept	Meaning / why it matters	Example	Interview angle
Actor	Policy distribution/deterministic action	Softmax actions or Gaussian mean	What parameters are optimized?
Critic	Estimates expected return	$V(s)$ or $Q(s, a)$	Critic is not a classifier
Advantage	Relative action quality	$Q(s, a) - V(s)$	Why lower variance?
TD error	One-step surprise	$r + \gamma V(s') - V(s)$	As advantage estimator
Baseline	State-dependent term that preserves expected policy gradient	Subtract $V(s)$	Why unbiased?
Shared encoder	Actor/critic may share representation	CNN trunk with two heads	Gradient interference trade-off
On/off-policy	Data from current policy or replay/other policy	A2C vs SAC	Corrections/stability

5. Algorithm / Working Process

For a basic one-step on-policy actor-critic:

1. Actor samples $a_t \sim \pi_\theta(\cdot \mid s_t)$.
2. Environment returns r_t , s_{t+1} , and termination flag.
3. Critic computes $V_w(s_t)$ and bootstrapped target $r_t + \gamma(1 - d_t)V_w(s_{t+1})$.
4. TD error δ_t trains critic toward the target.
5. Treat a detached δ_t as advantage and update actor to increase log-probability if positive and decrease it if negative.
6. Optionally add entropy, use n-step/GAE returns, batches, parallel environments, targets/replay, or continuous deterministic actors.
7. Evaluate with frozen parameters and a defined stochastic/deterministic action rule.

6. Mathematical Foundation

The policy-gradient theorem is

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s,a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi}(s, a)].$$

Subtracting any state-only baseline does not change its expectation, so use advantage:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

For a one-step value critic,

$$\delta_t = r_t + \gamma(1 - d_t)V_w(s_{t+1}) - V_w(s_t).$$

Losses can be

$$\mathcal{L}_{actor} = -\mathbb{E}[\log \pi_{\theta}(a_t | s_t) \text{stopgrad}(\delta_t)] - \beta \mathcal{H}(\pi),$$

$$\mathcal{L}_{critic} = \frac{1}{2} \mathbb{E}[\delta_t^2].$$

Detaching the advantage prevents the actor objective from changing critic predictions merely to reduce its own loss.

7. Practical Implementation

```
import torch
from torch import nn
import torch.nn.functional as F
from torch.distributions import Categorical

class ActorCritic(nn.Module):
    def __init__(self, obs_dim, actions):
        super().__init__()
        self.body = nn.Sequential(nn.Linear(obs_dim, 128), nn.Tanh())
        self.actor = nn.Linear(128, actions)
        self.critic = nn.Linear(128, 1)

    def forward(self, states):
        h = self.body(states)
        return Categorical(logits=self.actor(h)), self.critic(h).squeeze(-1)

model = ActorCritic(4, 2)
optimizer = torch.optim.Adam(model.parameters(), lr=3e-4)

states, next_states = torch.randn(32, 4), torch.randn(32, 4)
actions, rewards = torch.randint(0, 2, (32,)), torch.randn(32)
terminated = torch.zeros(32, dtype=torch.bool)

dist, values = model(states)
with torch.no_grad():
    _, next_values = model(next_states)
    targets = rewards + 0.99 * (~terminated).float() * next_values
    advantages = targets - values

actor_loss = -(dist.log_prob(actions) * advantages.detach()).mean()
critic_loss = F.mse_loss(values, targets)
```

```
loss = actor_loss + 0.5 * critic_loss - 0.01 * dist.entropy().mean()  
optimizer.zero_grad(); loss.backward(); optimizer.step()
```

8. Code Explanation

One shared body feeds a categorical actor and scalar value critic. The next-state value is computed without gradient because it is a bootstrap target. Actor log-probabilities are weighted by detached advantages. The critic fits returns, and entropy rewards non-collapsed action distributions. In longer rollouts, n-step returns or GAE replace one-step targets.

9. Training / Evaluation

Use vector environments, normalize observations, and distinguish terminal/truncated boundaries. Track actor loss carefully—it need not monotonically fall. Monitor return/success, critic loss, explained variance, entropy, KL/policy change, gradient norms, action statistics, and seed variance. Tune separate actor/critic LRs if needed, gamma, n-step horizon/GAE lambda, entropy coefficient, value coefficient, batch/rollout size, gradient clipping, and network sharing.

10. Complexity and Cost

On-policy actor-critic stores rollouts and requires policy/value evaluation per step plus training passes. Shared encoders reduce compute but couple gradients. Off-policy variants add replay and often target networks; they improve sample efficiency at extra memory/instability cost. Continuous actors output distribution parameters or deterministic actions, with cost scaling with action dimension.

11. Common Use Cases

Discrete/continuous control, robotics simulation, games, resource allocation, multi-agent systems, recurrent policies for partial observability, and policy optimization foundations such as PPO/SAC.

12. Common Mistakes

- Not detaching the advantage in a standard actor update
- Incorrect action log-probability, especially after tanh squashing
- Bootstrapping through true terminal states or cutting off at time limits
- Training critic on a target that still carries unintended gradients
- Mixing on-policy rollouts from very old policies without correction
- Reward leakage or exploitable reward shaping
- Assuming lower critic loss always means a better policy
- Using one seed and no deterministic evaluation protocol

13. Edge Cases / Limitations

A poor critic biases/noises the actor; a rapidly changing actor destabilizes the critic. Sparse/delayed rewards make credit assignment hard. Shared features can create conflicting gradients. On-policy forms are sample-inefficient; off-policy forms face extrapolation/bootstrapping instability. Partial observability requires memory or belief state.

14. Variations

- **A2C/A3C**: synchronous/asynchronous n-step actor-critic; placement foundations.
- **PPO**: clipped on-policy updates; most important practical baseline.
- **DDPG**: deterministic off-policy continuous control; historical/projects.
- **TD3**: twin critics, delayed actor, target smoothing reduce DDPG errors.
- **SAC**: entropy-regularized off-policy stochastic actor; strong continuous control.
- **IMPALA**: distributed actors with off-policy V-trace correction; large-scale systems.

15. Related Topics

Actor-critic vs REINFORCE: bootstrapped learned baseline lowers variance but can add bias. **Actor-critic vs DQN**: explicit differentiable policy supports continuous actions, while DQN derives a greedy discrete policy from Q-values. **A2C vs PPO**: direct advantage update versus clipped multiple-epoch reuse. **SAC vs PPO**: replay-based maximum-entropy off-policy learning versus clipped on-policy learning.

16. Interview Questions

1. **Why two components?** Actor chooses actions; critic estimates return quality to guide lower-variance updates.
2. **What does the critic output?** Depending on algorithm, $V(s)$, $Q(s, a)$, or related value distributions.
3. **What is advantage?** How much better an action is than the policy's average action at that state.
4. **Why does a baseline not bias policy gradient?** The expected score function times a state-only baseline is zero.
5. **Why detach advantage?** Actor optimization should change action probability, not manipulate the critic signal.
6. **What is TD error?** Reward plus discounted next value minus current value; a one-step advantage estimate.
7. **Bias-variance trade-off?** Monte Carlo returns have low bootstrap bias/high variance; short TD targets have more bias/lower variance.
8. **Can actor-critic handle continuous action?** Yes, with Gaussian/Beta policies or deterministic actors and suitable gradients.
9. **On-policy vs off-policy actor-critic examples?** A2C/PPO versus DDPG/TD3/SAC.
10. **Why entropy regularization?** It encourages exploration and prevents premature deterministic collapse.
11. **What if critic becomes inaccurate?** Actor follows misleading advantages; use better targets, normalization, capacity, learning-rate balance, and diagnostics.

17. Practice Tasks

Implement one-step actor-critic; replace TD(0) with n-step returns; compare REINFORCE variance with a critic baseline; add GAE; debug an undetached target; implement Gaussian actions with correct summed log-probability.

18. Project Ideas

Project	Function	Stack / environment	Resume value
Algorithm Ladder	Compares REINFORCE, A2C, PPO under fixed budgets	PyTorch, Gymnasium	Theory-to-experiment rigor
Robot Navigation	Learns discrete motion with partial observations	recurrent actor-critic, MiniGrid	Memory and reward shaping
Continuous Portfolio Simulator	Allocates bounded weights under costs	SAC/actor-critic, custom env	Continuous control and constraints

19. Quick Revision

- **Idea:** critic estimates advantage; actor follows it.
- **Formula:** $-\log \pi(a|s) * \text{stopgrad}(A)$ plus value loss.
- **Use:** learned stochastic/deterministic policies, including continuous actions.
- **Metrics:** return, success, entropy, explained variance, stability.
- **Trap:** gradient leakage through advantage/targets and terminal handling.
- **One-liner:** Actor-critic reduces policy-gradient variance by training a value estimator whose TD/advantage signal updates the policy.

20. Final Cheat Sheet

Item	Summary
Input / output	State $\rightarrow$ policy distribution/action and value estimate
Steps	Act $\rightarrow$ observe $\rightarrow$ value target/advantage $\rightarrow$ actor + critic update
Hyperparameters	Actor/critic LR, gamma, lambda/n-step, entropy/value weights
Pros / cons	Flexible, lower variance / coupled instability, reward sensitivity
Best use	Foundation for modern policy optimization/control

Cross-Topic Interview Map

If the requirement is...	Strong starting point	Why
Real-time object detection	YOLO	One-stage dense inference
Accuracy-first detection	Faster R-CNN	Proposal refinement and mature FPN baseline
Separate pixel mask per object	Mask R-CNN	Instance-level ROI mask head
Scalable pretrained vision representation	Vision Transformer	Patch-token Transformer scaling

If the requirement is...	Strong starting point	Why
Image-text retrieval or zero-shot classes	CLIP	Shared contrastive embedding space
General high-quality iterative generation	Diffusion Models	Flexible denoising framework
Practical text-to-image/editing	Stable Diffusion	Efficient conditional latent diffusion
Relational data, architecture undecided	GNN	General message-passing framework
Homophilous moderate node graph	GCN	Simple normalized convolution
New nodes and web-scale sampling	GraphSAGE	Inductive sampled aggregation
Unequal neighbor relevance	GAT	Learned edge attention
Discrete actions with replay	DQN	Off-policy value learning
Robust general policy-gradient baseline	PPO	Conservative clipped actor-critic updates
General policy + value architecture	Actor-Critic	Foundation for PPO, SAC, TD3, A2C